Master Bio-informatique

Module : Système et programmation de scripts

M. Belkhir

Email : mbelkhir@usthb.dz

Support de Cours

Année académique 2025-2026

Table des matières

Scripting Bash et automatisation de base **4**

I Fondamentaux du système Linux **5**
1 Définition d'un système d'exploitation 5
2 Historique de Linux 5
3 Architecture de linux 7
3.1 Applications utilisateur 7
3.2 Shell 8
3.3 Système d'exploitation 9
3.4 Couche 1 : Interface d'appel système 9

II La porte d'entrée vers Linux : Le shell **12**
1 Qu'est-ce qu'un shell ? 12
2 Différents Shells 13
3 Shell, Terminal, CMD et CLI : différences essentielles 13
4 Comment le shell interprète et exécute les commandes 14
5 Sessions et consoles virtuelles sous Linux 16
5.1 La session utilisateur sous Linux 16
5.2 Consoles virtuelles (Virtual Consoles) 16
5.3 Avantages des sessions et consoles virtuelles 16
6 Les types d'utilisateurs sous Linux 17
6.1 L'utilisateur ordinaire (user) 17
6.2 L'utilisateur superutilisateur (root) 17
6.3 les commandes `su` et `sudo` 17
7 Le manuel Linux 18
7.1 Organisation du manuel 18
7.2 Utilité du manuel 19
8 Quelques commandes de base 19
8.1 Introduction aux commandes de base 19

III Manipulation des fichiers **22**
1 Les fichiers sous Linux et l'organisation en arborescence 22
1.1 Le concept fondamental : « Tout est fichier » 22
1.2 Organisation des fichiers : une arborescence hiérarchique 22
2 Les commandes liées aux fichiers sous Linux 23
2.1 Commandes de navigation 23
2.2 Commandes de manipulation des fichiers et répertoires 24
3 Analyse détaillée du résultat de `ls -l` 25
3.1 1. Le type du fichier et les permissions 25

3.2 Nombre de liens (hard links) . . . 27
3.3 Propriétaire et groupe . . . 28
3.4 Taille du fichier . . . 28
3.5 Date et heure de dernière modification . . . 28
3.6 Nom du fichier . . . 28
3.7 Modifier les permissions : `chmod` . . . 29
4 Commandes avancées et expressions régulières . . . 30
4.1 Récapitulatif des symboles regex . . . 30
4.2 Commandes utilisant les expressions régulières . . . 31
5 Archivage et compression . . . 32
5.1 Commandes principales d'archivage . . . 33
5.2 Commandes de compression directe . . . 33

IV Programmation Shell **35**
1 Utiliser plusieurs commandes . . . 35
2 Créer un fichier script . . . 36
3 Les Variables dans le Shell . . . 37
3.1 Syntaxe et règles de nomination . . . 37
3.2 Deux types de variables . . . 38
3.3 Variables d'environnement . . . 38
3.4 Variables utilisateur . . . 39
3.5 Attributs des variables . . . 40
3.6 Lecture de valeurs avec `read` . . . 41
3.7 Les tableaux . . . 42
4 Calculs et Expressions dans le Shell . . . 43
4.1 Opérations avec `expr` . . . 43
5 Calculs avec l'arithmétique Bash . . . 44
6 Tests de Conditions dans le Shell . . . 45
6.1 Tests sur les valeurs numériques . . . 45
6.2 Tests sur les chaînes de caractères . . . 46
6.3 Tests sur les fichiers . . . 47
6.4 Syntaxe allégée . . . 47
7 Structures de contrôle . . . 47
7.1 Structure `if` . . . 48
7.2 Boucle `for` . . . 48
7.3 Boucle `while` . . . 48
7.4 Boucle `until` . . . 49
7.5 Structure `select` . . . 49
7.6 Structure `case` . . . 49
7.7 `break` et `continue` . . . 50
8 Les Fonctions en Shell . . . 50
8.1 Création d'une fonction Shell . . . 51
8.2 Utilisation des fonctions . . . 51
8.3 La valeur de retour d'une fonction . . . 51
8.4 Passage de paramètres aux fonctions . . . 52
8.5 Redéfinition et réorganisation des paramètres . . . 53
8.6 Variables locales et globales . . . 53

V Administration système **54**
1 Répertoires Système . . . 54
2 Gestion des Utilisateurs et Groupes . . . 55
2.1 Fichiers liées à la gestion des utilisateurs et groupes . . . 55
2.2 Création d'un Utilisateur . . . 56
2.3 Modification d'un Utilisateur . . . 57

VI Introduction à Python pour la Bioinformatique **59**
1 Pourquoi Python pour la bioinformatique ? . . . 59
2 Types de données en Python . . . 59
3 Variables . . . 60
4 Expressions et opérateurs . . . 60
4.1 Opérateurs numériques . . . 60
4.2 Précédence des opérateurs . . . 60
5 Instructions conditionnelles . . . 61
6 Boucles . . . 61
6.1 Boucle `while` . . . 61
6.2 Boucle `for` . . . 61
7 Chaînes de caractères . . . 61
8 Fonctions et Méthodes . . . 62
8.1 Fonctions intégrées courantes . . . 62
8.2 Méthodes courantes sur les chaînes de caractères . . . 62
8.3 Autres méthodes utiles . . . 63
9 Structures de données . . . 63
9.1 Listes . . . 63
9.2 Tuples . . . 63
9.3 Dictionnaires . . . 63
9.4 Ensembles . . . 64
10 Modules et Fichiers en Python . . . 64
10.1 Modules en Python . . . 64
10.2 Création d'un module . . . 65
10.3 Fichiers en Python . . . 65
10.4 Bonnes pratiques . . . 66
11 Documentation en Python . . . 66
11.1 La fonction `help()` . . . 66
11.2 Docstrings . . . 66
12 Exécution d'un script Python . . . 66
12.1 Exécution depuis le terminal . . . 67
12.2 Mode interactif . . . 67
12.3 Scripts exécutables sous Linux . . . 67
12.4 Environnements de développement . . . 67

Première partie

Scripting Bash et automatisation de base

Chapitre I

Fondamentaux du système Linux

Le système d'exploitation constitue le composant fondamental de tout ordinateur moderne. Il agit comme un intermédiaire entre le matériel et les programmes applicatifs, offrant un ensemble de services et de mécanismes permettant d'exécuter efficacement plusieurs tâches simultanément, de gérer les ressources matérielles et d'assurer la sécurité et la stabilité du système.

Linux, en particulier, est un exemple de système d'exploitation moderne qui combine robustesse, modularité et liberté logicielle. Ce chapitre a pour objectif de présenter les concepts de base nécessaires à la compréhension de Linux, en abordant son historique, son architecture en couches, l'espace utilisateur, le noyau et les caractéristiques essentielles qui font sa popularité.

1 Définition d'un système d'exploitation

Définition

les systèmes d'exploitation remplissent deux fonctions essentiellement distinctes :

- Fournir un ensemble abstrait et simplifié de ressources, plutôt que d'exposer directement le matériel complexe.
- Gérer les ressources matérielles elles-mêmes.

2 Historique de Linux

L'évolution des systèmes d'exploitation modernes, et en particulier de Linux, repose sur plusieurs étapes majeures qui ont façonné les concepts de multitâche, de portabilité et de liberté logicielle.

- **L'architecture de Von Neumann (1945)** Proposée par John von Neumann, cette architecture définit la structure fondamentale des ordinateurs modernes : une unité centrale (CPU) qui exécute les instructions, une mémoire unique pour les instructions et les données, et un flux séquentiel d'exécution. Cette organisation a permis de conceptualiser le fonctionnement des systèmes d'exploitation en tant qu'intermédiaires entre le matériel et les programmes applicatifs.

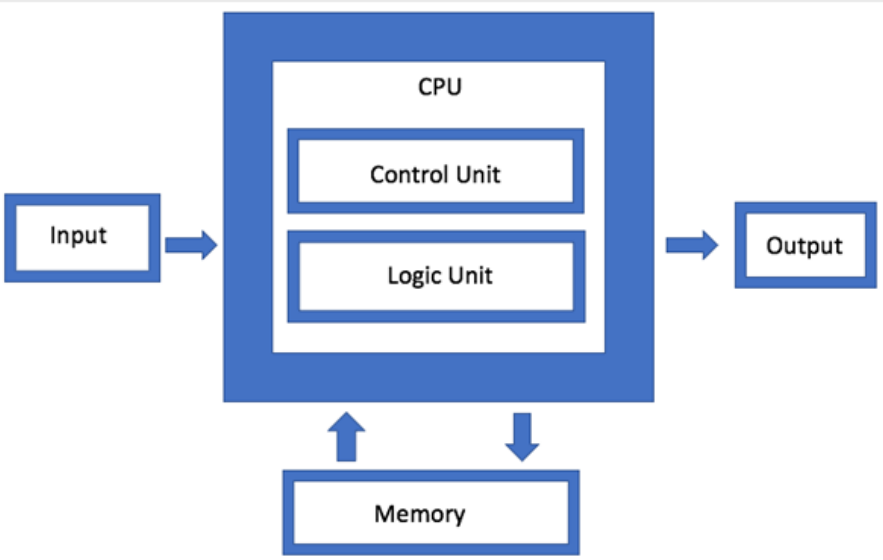


FIGURE I.1 – L'architecture de Von Neumann

— **Multics (1965)** Multics (Multiplexed Information and Computing Service) est un projet de système d'exploitation développé par MIT, General Electric et Bell Labs. Il introduit de nombreux concepts avancés pour l'époque : un système multitâche et multi-utilisateur, une hiérarchie de fichiers sécurisée, un contrôle d'accès fin, et un noyau modulaire. Bien que complexe et lourd, Multics a servi de modèle pour la conception de systèmes plus simples comme Unix.

— **Unix (1969)** Développé par Ken Thompson et Dennis Ritchie chez Bell Labs, Unix a été conçu pour être simple, portable et modulable. Il introduit :

  — Une structure hiérarchique des fichiers avec répertoires et sous-répertoires.
  — Des outils en ligne de commande (shell) permettant d'automatiser des tâches via des scripts.
  — Le concept de pipes et de redirection pour combiner des commandes.
  — Une philosophie "small is beautiful", favorisant des programmes simples et spécialisés pouvant être combinés.

  Unix devient la référence pour les systèmes multitâches modernes et influence directement le développement de Linux.

— **Le projet GNU (1983)** Initié par Richard Stallman, le projet GNU vise à créer un système d'exploitation **libre et complet**. Il fournit des outils essentiels tels que le compilateur GCC, l'éditeur Emacs, et de nombreuses bibliothèques. Cependant, il manque encore un **noyau libre** capable de gérer le matériel directement.

— **Linux (1991)** Linus Torvalds développe le noyau Linux à l'université d'Helsinki. Il s'agit d'un noyau multitâche, préemptif et libre, compatible avec la philosophie Unix. Combiné aux outils GNU, Linux devient un **système d'exploitation complet et libre**, capable de tourner sur une grande variété de matériel. Sa modularité, sa robustesse et sa portabilité en font le socle de nombreux systèmes modernes, allant des serveurs aux superordinateurs.

Conclusion

Linux est le résultat d'une longue évolution historique, combinant : l'architecture matérielle de Von Neumann, les concepts avancés de Multics, la simplicité et la philosophie Unix, ainsi que les outils libres du projet GNU.

3 Architecture de linux

La conception des ordinateurs modernes et des systèmes d'exploitation repose sur un modèle emboîté en couches. Ce modèle permet de structurer l'architecture d'un ordinateur depuis les circuits électroniques les plus bas le matériel jusqu'aux applications utilisées par l'utilisateur final. Dans le contexte d'un système comme Linux, on peut en simplifier la représentation selon les niveaux suivants : utilisateur (via des applications), interface utilisateur (shell ou GUI), noyau, puis matériel.

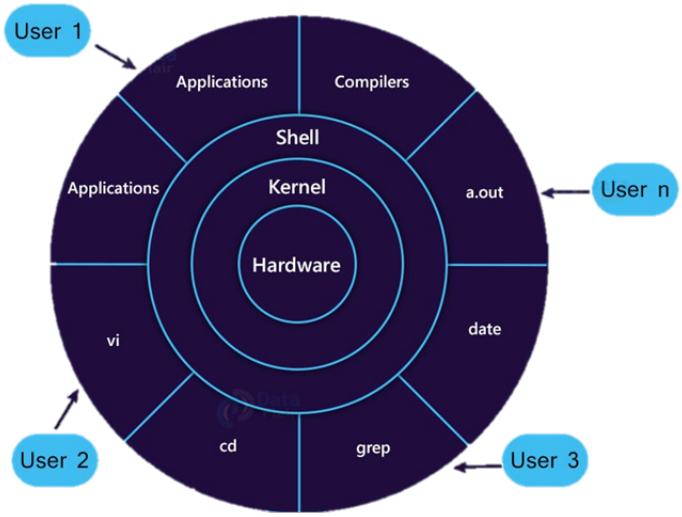


FIGURE I.2 – Architecture de Linux

3.1 Applications utilisateur

Dans l'architecture d'un système d'exploitation GNU/Linux, l'espace utilisateur (*user space*) constitue la partie supérieure du système, séparée du noyau afin d'assurer isolation, sécurité et stabilité. Cet espace regroupe l'ensemble des programmes et bibliothèques avec lesquels l'utilisateur ou les processus interagissent directement. On peut distinguer donc deux composants essentiels dans l'espace utilisateur : les applications et les bibliothèques.

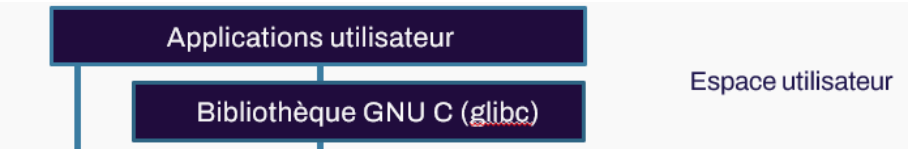


FIGURE I.3 – Espace utlisateur

3.1.1 Les applications utilisateur

Les applications représentent tous les programmes exécutés par l'utilisateur ou par d'autres processus : éditeurs de texte, navigateurs Web, interpréteurs de commandes, outils systèmes, environnements graphiques, etc. Ces programmes n'interagissent pas directement avec le matériel ni avec le noyau ; ils s'appuient sur les bibliothèques pour invoquer les services du système (création de fichiers, gestion de mémoire, communications réseau, etc.).

Les applications sont exécutées en *mode utilisateur*, un mode d'exécution restreint qui empêche l'accès direct au matériel. Cette séparation permet de protéger le système contre les erreurs applicatives potentielles, qui ne peuvent pas compromettre la stabilité du noyau.

3.1.2 Les bibliothèques GNU

Parmi les composants centraux du projet GNU figure la **GNU C Library** (`glibc`), qui constitue la bibliothèque standard du langage C dans la plupart des systèmes Linux.

La bibliothèque `glibc` joue un rôle crucial comme intermédiaire entre les applications et le noyau. Elle fournit :

— l'ensemble des fonctions standard du langage C (`printf()`, `malloc()`, `fopen()`, etc.) ;

— des API de plus haut niveau pour la gestion des processus, des fichiers et des communications inter-processus ;

— une interface uniforme vers les appels système (system calls), permettant aux applications d'invoquer les services du noyau sans manipuler directement des instructions bas niveau.

Ainsi, `glibc` fait office de couche d'abstraction qui simplifie le développement et renforce la portabilité des programmes. Lorsqu'une application a besoin d'interagir avec le noyau (par exemple pour ouvrir un fichier ou allouer de la mémoire), ce sont les fonctions de `glibc` qui traduisent ces demandes en appels système adaptés à la plateforme sous-jacente.

3.2 Shell

Le shell joue le rôle d'*interface entre l'utilisateur (ou ses applications) et le noyau*. Il permet :

— d'accepter les commandes de l'utilisateur (via clavier/terminal ou interface graphique).

— d'envoyer les requêtes correspondantes au noyau via des appels système pour effectuer les actions demandées (lecture/écriture de fichiers, lancement de programmes, gestion des processus, etc.).

— de fournir des facilités supplémentaires.

Le shell sera détaillé dans le chapitre suivant.

3.3 Système d'exploitation

L'architecture d'un système d'exploitation de type Unix comme GNU/Linux peut être présentée sous la forme de trois couches principales : l'interface d'appel système, le noyau et le code dépendant de l'architecture. Cette organisation clarifie le rôle de chaque composant et la manière dont ils coopèrent pour assurer le fonctionnement du système.

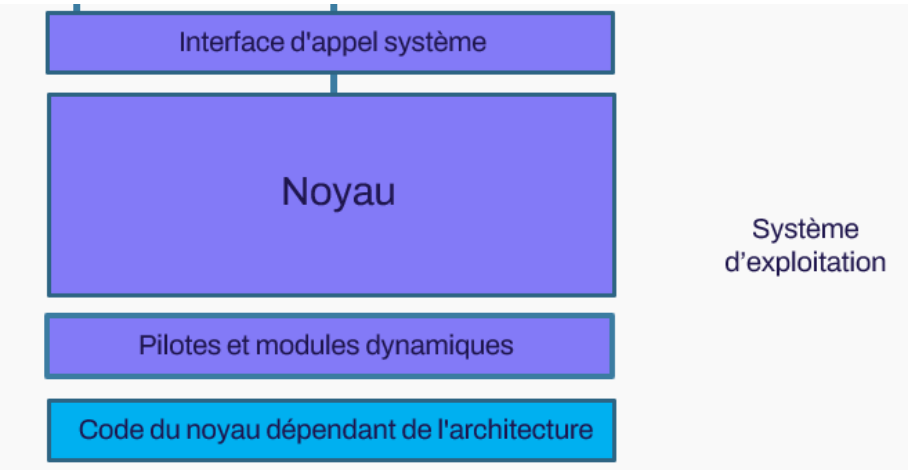


FIGURE I.4 – Système d'exploitation

3.4 Couche 1 : Interface d'appel système

L'*Interface d'appel système* (System Call Interface – SCI) constitue la frontière entre l'espace utilisateur et le noyau. Elle est l'unique point d'entrée permettant aux applications et aux bibliothèques utilisateur (notamment `glibc`) d'accéder aux services du système d'exploitation.

Les appels système tels que `read()`, `write()`, `open()`, `fork()` ou `socket()` fournissent un accès abstrait, standardisé et sécurisé aux ressources matérielles. Cette couche joue un rôle fondamental en :

— empêchant les applications d'accéder directement au matériel ;

— assurant la protection et l'isolation des processus ;

— garantissant la compatibilité et la portabilité du code applicatif.

3.4.1 Sous-système processus

Le sous-système processus est un élément central du noyau Linux. Il gère la création, l'exécution, la synchronisation et la terminaison des programmes, tout en garantissant à chaque processus un espace mémoire isolé via la mémoire virtuelle et la pagination.

Il fournit également des mécanismes de communication et de coordination (IPC) tels que pipes, sémaphores, mémoire partagée, signaux et sockets, permettant aux processus d'échanger des données localement ou via le réseau. La pile réseau du noyau gère TCP/IP et les interfaces, étendant ainsi l'IPC à la communication distribuée.

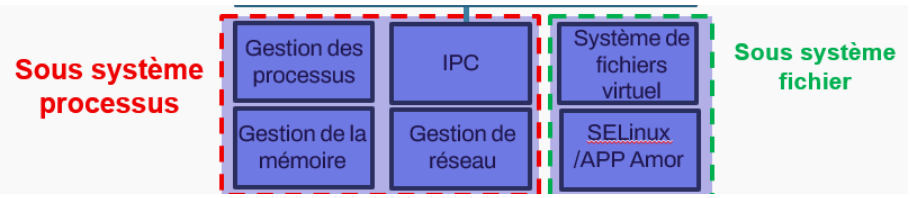


FIGURE I.5 – Noyau

a) **La gestion des processus** : Le noyau crée, détruit et supervise les processus du système. Dans Linux, processus et threads sont représentés de manière unifiée à travers la structure appelée *task*. L'ordonnancement est assuré par un ensemble d'algorithmes hiérarchiques fonctionnant avec plusieurs niveaux de priorité, permettant de gérer divers types de charges : processus interactifs, activités intensives en calcul, tâches temps réel, etc. Le but est de garantir une exécution fluide, équitable et réactive.

b) **La gestion de la mémoire** : Chaque processus dispose d'un **espace d'adressage virtuel** distinct, offrant l'illusion d'une mémoire propre et continue. Grâce à la **pagination**, la mémoire virtuelle permet de séparer les adresses utilisées par les programmes de la mémoire physique réelle. Le noyau s'occupe :

   — d'allouer et libérer les pages mémoire nécessaires,
   — de maintenir l'isolation entre processus,
   — de sélectionner intelligemment les pages à conserver ou à déplacer lors des situations de forte charge mémoire.

   Ce mécanisme assure efficacité, isolation et stabilité globale du système.

c) **La communication interprocessus (IPC)** : Les processus doivent échanger des données ou se synchroniser afin de fonctionner correctement dans un environnement multiprogrammé. Linux propose un ensemble complet de mécanismes IPC : pipes, files de messages, sémaphores, mémoire partagée, signaux, ainsi que les sockets permettant la communication locale ou réseau. Ces outils assurent la coordination entre processus, que ceux-ci se trouvent sur la même machine ou sur des systèmes distants.

d) **Gestion de réseau** : Le noyau implémente une pile réseau complète intégrant les principaux protocoles (TCP, UDP, IP) ainsi que les interfaces physiques (Ethernet, Wi-Fi). Ce sous-ensemble gère également le routage, le filtrage, la gestion des connexions et le transfert de paquets. Il fournit ainsi aux applications une abstraction uniforme pour communiquer à travers un réseau local ou l'Internet.

3.4.2 Couche 3 : Code dépendant de l'architecture (BSP)

Bien que le noyau soit conçu pour être portable, une partie du code reste spécifique à chaque architecture matérielle. Ce code, souvent écrit en assembleur, est regroupé dans des *Board Support Packages* (BSP). Il permet au noyau générique de fonctionner sur une grande variété de plateformes.

Ce code dépendant de l'architecture comprend :

— **les pilotes spécifiques au processeur et aux périphériques** (gestion des registres, modes d'exécution) ;
— **les routines de démarrage** initialisant le processeur, la mémoire et les registres avant l'exécution du noyau en C ;

— **la gestion des interruptions matérielles et des horloges** (contrôleurs APIC, GIC, timers) ;

— **les paramètres liés à la carte mère ou à la plateforme** (mapping mémoire, topologie, bus, matériel embarqué).

Cette couche assure le lien entre le matériel et le noyau portable, en masquant les particularités des architectures sous des interfaces unifiées.

Conclusion

Linux possède plusieurs caractéristiques essentielles qui expliquent sa popularité et son usage répandu :

— **Modularité** : Le noyau peut être étendu ou réduit par le chargement dynamique de modules, permettant d'adapter le système aux besoins spécifiques.

— **Open source** : Le code source est librement accessible et modifiable. Cette transparence permet aux développeurs et aux experts en sécurité d'auditer, corriger et améliorer continuellement le système, contribuant directement à sa sécurité.

— **Sécurité** : Isolation des processus, contrôle des accès et intégration de mécanismes avancés (SELinux, AppArmor) pour protéger le système.

— **Portabilité** : Fonctionne sur une grande variété d'architectures matérielles grâce à l'abstraction du matériel et aux modules BSP.

— **Multiutilisateur** : Permet à plusieurs utilisateurs de travailler simultanément tout en isolant leurs ressources.

— **Multitâche** : Exécution concurrente de multiples processus avec un ordonnanceur efficace garantissant performance et réactivité.

Chapitre II

La porte d'entrée vers Linux : Le shell

1 Qu'est-ce qu'un shell ?

Définition

Le **shell** est l'interface principale entre l'utilisateur et le système d'exploitation. Il s'agit d'un *interpréteur de commandes* chargé de recevoir les instructions de l'utilisateur, de les analyser et de demander au noyau d'exécuter les opérations correspondantes.

Le shell peut fonctionner de deux manières :

— **interactivement**, lorsque l'utilisateur saisit directement des commandes (par exemple `bash`, `zsh`) ;

— **non interactivement**, lors de l'exécution automatique de scripts.

Il fournit plusieurs services essentiels, parmi lesquels :

— l'exécution des programmes ;

— la gestion des fichiers et des processus ;

— les redirections et les tubes (*pipes*) ;

— l'automatisation des tâches via des scripts ;

— la configuration de l'environnement utilisateur.

Pourquoi utiliser le terminal plutôt qu'une interface graphique ? Le terminal permet d'interagir directement avec le système via des commandes textuelles, offrant un contrôle plus précis et plus rapide que la plupart des interfaces graphiques. Historiquement, Unix est apparu avant l'existence des interfaces graphiques, et les utilisateurs devaient donc se contenter du terminal pour exécuter toutes les opérations. Aujourd'hui encore, le terminal reste essentiel pour automatiser des tâches avec des scripts, gérer des systèmes à distance, et accéder à des fonctionnalités avancées souvent indisponibles dans les interfaces graphiques. De plus, il est léger, consomme peu de ressources et fonctionne même sur des systèmes dépourvus d'environnement graphique, ce qui le rend indispensable pour les administrateurs système, les développeurs et les utilisateurs avancés souhaitant efficacité et flexibilité.

2 Différents Shells

Au fil de l'évolution des systèmes UNIX et GNU/Linux, plusieurs shells ont été développés. Bien qu'ils remplissent tous la même fonction servir d'interpréteur de commandes ils diffèrent par leurs fonctionnalités, leur syntaxe et leurs outils intégrés.

a) **Bourne Shell (sh)** Le **sh** est l'un des premiers shells UNIX. Simple et robuste, il a servi de base à de nombreux shells modernes. Il reste utilisé pour l'écriture de scripts garantissant une compatibilité maximale.

b) **Bash (Bourne Again SHell)** Le **bash** est le shell par défaut sur la plupart des distributions Linux. Il apporte de nombreuses améliorations : historique de commandes, complétion automatique, gestion avancée des scripts et variables.

c) **C Shell (csh) et Tcsh** Le **csh** propose une syntaxe inspirée du langage C. Sa version améliorée, **tcsh**, ajoute la complétion, l'édition de ligne et une meilleure gestion interactive.

d) **Korn Shell (ksh)** Le **ksh** combine les fonctionnalités du Bourne Shell avec des extensions inspirées du C Shell. Il est apprécié pour ses performances et ses capacités de script avancées.

e) **Z Shell (zsh)** Le **zsh** est un shell moderne intégrant des fonctionnalités avancées : complétion intelligente, correction automatique, puissants mécanismes de personnalisation. Il est souvent utilisé dans des environnements interactifs.

f) **Fish (Friendly Interactive Shell)** Le **fish** privilégie la convivialité : coloration syntaxique, suggestions automatiques, configuration simple. Il n'est pas strictement POSIX (la norme qui définit le comportement standard des shells et outils UNIX), mais offre une expérience utilisateur très intuitive.

3 Shell, Terminal, CMD et CLI : différences essentielles

Pour interagir avec un ordinateur, il existe plusieurs moyens. L'un des plus puissants et flexibles est l'utilisation d'une interface textuelle, où l'utilisateur tape des commandes pour contrôler le système.

Dans ce contexte, plusieurs notions sont importantes : le **shell**, le **terminal**, le **CMD** et la **CLI** (Command Line Interface). Chacun a un rôle spécifique :

1. **Shell** est un programme chargé d'interpréter les commandes saisies par l'utilisateur. Lorsqu'une commande est entrée, le shell l'analyse, vérifie sa validité et demande ensuite au noyau d'exécuter l'action correspondante.
2. **Terminal** n'interprète aucune commande ; il se contente d'afficher une interface texte permettant à l'utilisateur de saisir des instructions. Il héberge simplement un shell. Ainsi, il est possible de changer de terminal sans changer de shell, et inversement.

   Exemple

   Exemples de terminaux : `gnome-terminal`, `konsole`, `xterm` ou `Windows Terminal`.

3. **CMD** est l'interpréteur de commandes historique de Windows. Il remplit un rôle similaire à celui d'un shell, mais avec une syntaxe propre à l'environnement Windows

et sans compatibilité POSIX. Moins puissant que les shells Unix modernes, il n'est généralement pas adapté à l'écriture de scripts portables.

4. **CLI (Command Line Interface)** une interface textuelle permettant d'interagir avec un programme au moyen de commandes. Le shell est un type particulier de CLI, mais de nombreux autres outils en proposent également : Git (`git`), l'interpréteur Python (`python`), ou encore le client MySQL (`mysql`).

Le tableau II.1 résume les differences.

Terme	**Rôle principal**	**Nature**
Shell	Interprète les commandes	Interpreteur/Langage de script
Terminal	Héberge le Shell, affiche-les entrées/sorties	Application d'affichage
CMD	Shell Windows classique	Langage spécifique
CLI	Manière d'interagir par texte	Concept

TABLE II.1 – Différences entre Shell, Terminal, CMD et CLI

4 Comment le shell interprète et exécute les commandes

Le schéma en Figure II.1 décrit le processus complet qu'un shell (interpréteur de commandes) suit pour exécuter chaque commande saisie par l'utilisateur. Ce processus peut être décomposé en plusieurs étapes fondamentales :

Étape 1 : Lecture et Analyse Syntaxique

— **Lecture de la commande** : Le shell commence par lire la commande saisie par l'utilisateur au clavier.

— **Analyse syntaxique** : Le shell analyse la structure de la commande pour vérifier sa validité syntaxique.

  — **Avec erreur** : Si une erreur syntaxique est détectée, le shell affiche un message d'erreur approprié.

  — **Sans erreur** : Si la syntaxe est correcte, le processus continue.

Étape 2 : Identification du Type de Commande

Le shell détermine ensuite si la commande est :

— **Commande interne** : Une fonction intégrée directement au shell lui-même (comme `cd`, `echo`, `exit`).

— **Commande externe** : Un programme exécutable indépendant situé dans le système de fichiers (comme `ls`, `grep`, `cat`).

Étape 3 : Exécution de la Commande

Le processus d'exécution diffère selon le type de commande :

Pour les commandes internes :

— Le shell exécute directement le code correspondant depuis sa mémoire interne.

— Aucun processus supplémentaire n'est créé.

Pour les commandes externes :

— **Fork** : Le shell crée un nouveau processus fils identique au processus père.

— **Exécution** : Le processus fils se transforme en exécutant le programme correspondant.

— **Attente** : Le shell père attend que le processus fils termine son exécution.

— **Signal de fin** : Le processus fils signale sa terminaison au processus père.

Étape 4 : Retour au Prompt

Après l'exécution (qu'elle soit réussie ou non), le shell retourne à l'état d'attente et affiche à nouveau le prompt, prêt à recevoir une nouvelle commande.

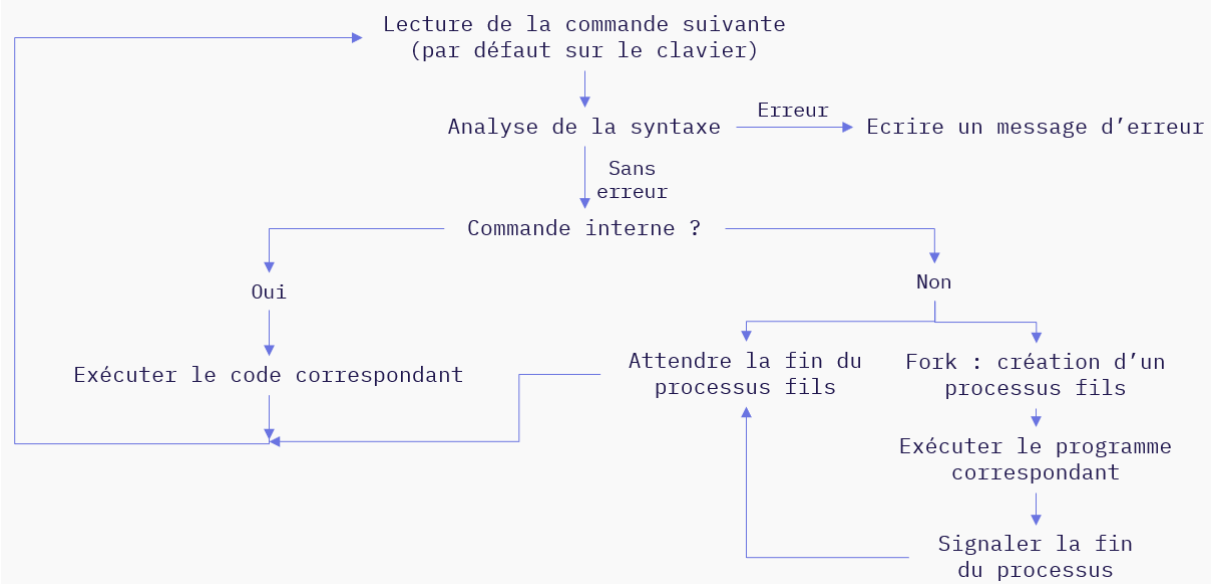


FIGURE II.1 – fonctionnement du shell

Importance de ce Mécanisme

Ce processus assure plusieurs fonctions cruciales :

— **Stabilité** : En exécutant les commandes externes dans des processus séparés, le shell principal est protégé contre les plantages des programmes.

— **Isolation** : Chaque commande externe s'exécute dans son propre espace mémoire.

— **Contrôle** : Le shell peut gérer plusieurs processus et attendre leur terminaison.

— **Flexibilité** : Le mécanisme permet à la fois l'exécution rapide des commandes internes et l'exécution sécurisée des programmes externes.

5 Sessions et consoles virtuelles sous Linux

Linux offre la possibilité d'utiliser plusieurs **sessions** et **consoles virtuelles** afin de gérer l'accès des utilisateurs au système de manière efficace et sécurisée. Chaque session constitue un environnement isolé pour un utilisateur, incluant ses processus, fichiers et variables d'environnement, garantissant que les actions d'un utilisateur n'affectent pas celles des autres. Les consoles virtuelles, quant à elles, permettent de disposer de plusieurs terminaux textuels indépendants sur la même machine physique. Grâce à elles, plusieurs utilisateurs peuvent se connecter simultanément, ou un même utilisateur peut gérer plusieurs tâches parallèles sur différentes consoles. Ce mécanisme est particulièrement utile pour l'administration système, le dépannage, l'exécution de processus à distance, ou encore pour maintenir la continuité du travail en laissant des sessions actives sur différentes consoles.

5.1 La session utilisateur sous Linux

Une **session utilisateur** correspond à l'ensemble des ressources et processus associés à un utilisateur depuis le moment où il se connecte jusqu'à sa déconnexion. Elle constitue l'environnement personnel de l'utilisateur et permet d'exécuter des applications, de gérer des fichiers et d'interagir avec le système.

Caractéristiques principales d'une session :

- **Isolation** : chaque session est séparée des autres, garantissant que les processus et fichiers d'un utilisateur ne perturbent pas ceux des autres.
- **Variables d'environnement** : chaque session possède ses propres variables (comme `PATH`, `HOME`, `SHELL`) qui définissent le comportement des programmes.
- **Gestion des processus** : tous les processus lancés par l'utilisateur appartiennent à sa session, facilitant le contrôle et la terminaison des processus.
- **Durée de vie** : une session commence à la connexion de l'utilisateur et se termine à sa déconnexion ou à la fermeture de la console.
- **Accès local ou distant** : la session peut être ouverte directement sur la machine physique ou via des protocoles à distance comme `SSH`.

5.2 Consoles virtuelles (Virtual Consoles)

Les **consoles virtuelles** permettent de disposer de plusieurs terminaux textuels indépendants sur la même machine physique. Sous Linux, elles sont accessibles généralement via les combinaisons de touches `Ctrl+Alt+F1` à `Ctrl+Alt+F6` (ou plus selon la configuration). Chaque console virtuelle peut accueillir une session distincte et offrir un accès simultané à plusieurs utilisateurs sur le même ordinateur.

5.3 Avantages des sessions et consoles virtuelles

- Permettent à plusieurs utilisateurs de travailler simultanément sur le même système.
- Offrent une séparation et une isolation entre les environnements utilisateur.
- Facilitent l'administration et le dépannage, car chaque console peut être utilisée indépendamment.

— Maintiennent la continuité du travail : un utilisateur peut laisser une session active dans une console pendant qu'un autre utilise une autre console.

Ainsi, la combinaison des sessions et des consoles virtuelles constitue un mécanisme fondamental de Linux pour le multitâche et le multi-utilisateur, tout en garantissant sécurité, isolation et flexibilité.

6 Les types d'utilisateurs sous Linux

Linux distingue principalement deux catégories d'utilisateurs, chacune ayant des droits et responsabilités différents sur le système :

6.1 L'utilisateur ordinaire (user)

— Il s'agit des comptes créés pour des personnes utilisant le système au quotidien.

— Ces utilisateurs disposent de droits limités afin de protéger le système et les fichiers des autres utilisateurs.

— Ils peuvent exécuter des programmes, créer et modifier leurs propres fichiers, mais ne peuvent pas installer de logiciels ou modifier les configurations système critiques.

— Dans un terminal, le prompt d'un utilisateur standard est généralement indiqué par `$`.

Exemple

un étudiant, un employé ou tout utilisateur standard d'un poste Linux.

6.2 L'utilisateur superutilisateur (root)

— Le **superutilisateur** ou **root** possède tous les droits sur le système.

— Il peut créer, modifier ou supprimer des comptes utilisateurs, installer des logiciels, configurer le système et accéder à tous les fichiers.

— L'utilisation de ce compte nécessite la prudence : une erreur peut affecter l'ensemble du système.

— Pour des raisons de sécurité, les administrateurs utilisent souvent le compte root via des commandes spécifiques (`sudo`) plutôt que de se connecter directement.

— Dans un terminal, le prompt du superutilisateur est généralement indiqué par #.

6.3 les commandes `su` et `sudo`

Dans Linux, certaines opérations nécessitent des privilèges élevés, comme installer des logiciels, modifier des fichiers système ou configurer des services. Pour effectuer ces actions sans compromettre la sécurité globale du système, on utilise des mécanismes permettant d'obtenir temporairement les droits du superutilisateur :

— `su` **(substitute user)** : cette commande permet de basculer complètement vers un autre utilisateur, généralement root. Elle ouvre une nouvelle session shell avec tous les privilèges de l'utilisateur cible.

— `sudo` **(superuser do)** : cette commande permet d'exécuter une seule commande avec les privilèges du superutilisateur, sans ouvrir une session complète. Elle limite les risques et offre une traçabilité grâce aux journaux d'utilisation.

Ces deux outils sont essentiels pour gérer le système de manière sécurisée, car ils permettent de contrôler l'accès aux privilèges administratifs et d'éviter de rester connecté en permanence avec le compte root, ce qui pourrait entraîner des erreurs critiques ou des failles de sécurité.

Commande	**Fonction principale**	**Caractéristiques**
`su`	Permet de devenir le superutilisateur ou un autre utilisateur	Nécessite le mot de passe de l'utilisateur cible ; ouvre une nouvelle session shell ; donne un contrôle complet jusqu'à déconnexion
`sudo`	Permet d'exécuter une seule commande avec les privilèges du superutilisateur	Utilise le mot de passe de l'utilisateur courant ; ne change pas la session ; limite le scope de la commande ; journalise l'utilisation pour la sécurité

TABLE II.2 – Comparaison entre `su` et `sudo`

7 Le manuel Linux

Le système Linux fournit une documentation intégrée très complète appelée **manuel** ou `man`. Le manuel permet aux utilisateurs et administrateurs d'accéder rapidement à la description des commandes, des fonctions système, des fichiers de configuration et des programmes.

7.1 Organisation du manuel

Le manuel Linux est organisé en **sections numérotées**, chacune correspondant à un type de contenu particulier :

— **Section 1 :** Commandes utilisateur (programmes exécutables)
— **Section 2 :** Appels système (system calls)
— **Section 3 :** Fonctions de bibliothèque (C library functions)
— **Section 4 :** Fichiers spéciaux et périphériques
— **Section 5 :** Formats de fichiers et conventions
— **Section 6 :** Jeux et démonstrations
— **Section 7 :** Divers, standards, conventions et macros
— **Section 8 :** Commandes et utilitaires d'administration système

Chaque page du manuel contient généralement :

— le nom de la commande ou fonction ;

— une description de son rôle ;
— la syntaxe complète ;
— les options et arguments disponibles ;
— des exemples d'utilisation ;
— parfois des informations sur les fichiers de configuration associés.

7.2 Utilité du manuel

Le manuel Linux est un outil essentiel pour :
— comprendre le rôle et le fonctionnement d'une commande ou d'une fonction ;
— apprendre les options disponibles et leur syntaxe correcte ;
— résoudre des problèmes ou configurer le système correctement ;
— se référer à la documentation officielle de manière rapide et fiable.

Accès au manuel Pour consulter une page du manuel, on utilise la commande :

Commande : `man`

Définition / Rôle : Affiche la page du manuel relative à la commande.
Syntaxe : `man [section] nom commande`
Exemple : `man ls`

Conseil pédagogique Le manuel est accessible directement depuis le terminal et constitue un outil incontournable pour tous les utilisateurs Linux, débutants comme avancés. Il favorise l'autonomie et l'apprentissage progressif du système, tout en offrant une **documentation fiable et officielle**, régulièrement mise à jour avec le système et les commandes.

8 Quelques commandes de base

8.1 Introduction aux commandes de base

Linux offre une grande variété de commandes pour interagir avec le système, gérer les fichiers, consulter l'état des processus, ou encore obtenir des informations sur l'environnement. Pour un utilisateur débutant, il est important de comprendre non seulement la syntaxe des commandes, mais aussi leur rôle et leur contexte d'utilisation.

Les commandes qui seront présentées dans ce support constituent un ensemble fondamental, offrant les outils essentiels pour naviguer dans le système, gérer les fichiers, interagir avec les processus et exploiter efficacement l'environnement Linux.

Chaque commande est présentée selon un format uniforme pour faciliter l'apprentissage et la lecture :

1. **Commande** : nom de la commande ;
2. **Définition / Rôle** : description de la fonction principale de la commande ;
3. **Syntaxe** : la manière générique d'utiliser la commande ;
4. **Exemple** : illustration pratique d'utilisation.

Remarque : Ce format sera utilisé de manière cohérente tout au long de ce support de cours pour toutes les commandes Linux présentées, afin d'assurer clarté, homogénéité et facilité d'apprentissage pour l'utilisateur.

Commande : `apropos`

Définition / Rôle : Recherche dans les pages du manuel les mots-clés correspondant à un terme.
Syntaxe : `apropos [mot_clé]`
Exemple : `apropos passwd`

Commande : `whatis`

Définition / Rôle : Fournit une courte description d'une commande ou d'une fonction.
Syntaxe : `whatis [commande]`
Exemple : `whatis ls`

Commande : `whoami`

Définition / Rôle : Affiche le nom de l'utilisateur actuellement connecté.
Syntaxe : `whoami`
Exemple : `whoami`

Commande : `id`

Définition / Rôle : Affiche l'identité de l'utilisateur et les groupes auxquels il appartient.
Syntaxe : `id [utilisateur]`
Exemple : `id`

Commande : `pwd`

Définition / Rôle : Affiche le chemin complet du répertoire courant.
Syntaxe : `pwd`
Exemple : `pwd`

Commande : `ls`

Définition / Rôle : Liste le contenu d'un répertoire.
Syntaxe : `ls [options] [fichier/répertoire]`
Exemple : `ls -l /home/user`

Commande : `cd`

Définition / Rôle : Change le répertoire courant.
Syntaxe : `cd [répertoire]`
Exemple : `cd /home/user/Documents`

Commande : `cd ..`

Définition / Rôle : Remonte d'un niveau dans l'arborescence des répertoires.
Syntaxe : `cd ..`
Exemple : `cd ..`

Commande : `date`

Définition / Rôle : Affiche ou modifie la date et l'heure du système.
Syntaxe : `date [options]`
Exemple : `date "+%Y-%m-%d %H:%M:%S"`

Commande : `uname`

Définition / Rôle : Affiche des informations sur le système d'exploitation et le noyau.
Syntaxe : `uname [options]`
Exemple : `uname -a`

Commande : `history`

Définition / Rôle : Affiche l'historique des commandes saisies dans le shell.
Syntaxe : `history [n]`
Exemple : `history`

Commande : `clear`

Définition / Rôle : Efface le contenu affiché du terminal.
Syntaxe : `clear`
Exemple : `clear`

Conclusion

— Le Shell est le cœur de l'interaction avec Linux.
— Il permet de contrôler, comprendre et automatiser le système.

Chapitre III

Manipulation des fichiers

1 Les fichiers sous Linux et l'organisation en arborescence

1.1 Le concept fondamental : « Tout est fichier »

Sous Linux, tout est représenté sous forme de **fichier**. Ce principe constitue l'un des fondements des systèmes UNIX et implique que :

- les documents classiques (texte, images, programmes) sont des fichiers ;
- les périphériques (clavier, disque, carte réseau) sont accessibles comme des fichiers dans `/dev` ;
- les processus exposent leurs informations via des fichiers (ex. `/proc`) ;
- les sockets, tubes nommés, liens symboliques sont également représentés comme des fichiers spéciaux.
- Tous ces fichiers sont identifiés de manière unique par un **descripteur de fichier** (File Descriptor) et sont représentés par une structure de données interne appelée **inode**, qui contient toutes les informations relatives au fichier : type, permissions, propriétaire, taille, emplacements des blocs sur le disque, dates de création/modification, etc.

Cette vision unifiée permet une grande simplicité dans le développement et l'administration du système : si tout est fichier, alors toutes les interactions reposent sur des opérations simples comme `ouvrir`, `lire`, `écrire` ou `fermer`.

1.2 Organisation des fichiers : une arborescence hiérarchique

Tous les fichiers et répertoires de Linux sont organisés dans une structure hiérarchique appelée **arborescence**. Cette arborescence est représentée sous la forme d'un arbre inversé, dont le point de départ est la racine du système : le répertoire `/` (appelé « root »).

Voici les principales caractéristiques de cette organisation :

1. Une racine unique

L'arborescence Linux possède **une seule racine** : le répertoire `/`. À partir de cette racine, tous les autres répertoires se déploient. Il n'existe donc pas plusieurs points de

départ, contrairement à Windows où chaque partition possède une lettre (ex. `C:`, `D:`, etc.).

2. Une structure sans cycles

L'arborescence est une vraie structure d'arbre :

— aucun répertoire ne peut contenir directement ou indirectement l'un de ses ancêtres ;

— il est donc impossible de former une boucle (cycle).

Cette propriété garantit la cohérence de la navigation dans le système.

3. Un chemin unique pour chaque fichier

Chaque fichier ou répertoire du système possède un **chemin unique** qui permet de l'identifier sans ambiguïté. Ce chemin peut être :

— **absolu** : il commence toujours par `/`, par exemple `/home/user/Documents/note.txt`

— **relatif** : il est exprimé par rapport au répertoire courant.

Ce principe assure une référence claire et précise à n'importe quel élément du système.

2 Les commandes liées aux fichiers sous Linux

Dans Linux, l'interaction avec les fichiers et répertoires passe principalement par deux familles de commandes :

— **les commandes de navigation** : elles permettent de se déplacer dans l'arborescence, d'afficher le contenu des répertoires et de consulter des fichiers ;

— **les commandes de manipulation** : elles permettent de créer, copier, déplacer, renommer ou supprimer des fichiers et répertoires.

La maîtrise de ces commandes est essentielle pour comprendre et exploiter efficacement le système Linux. Les commandes ci-dessous suivent le format standard défini dans ce support : **commande**, **rôle**, **syntaxe**, **exemple**.

2.1 Commandes de navigation

En complément des commandes fondamentales déjà introduites dans le chapitre précédent telles que ls, cd et pwd nous présentons ci-dessous une série de commandes supplémentaires (liste non exhaustive) permettant d'approfondir la navigation et la manipulation des fichiers sous Linux.

Commande : `cat`

Définition / Rôle : Affiche le contenu d'un fichier ou concatène plusieurs fichiers.
Syntaxe : `cat [options] fichier`
Exemple : `cat notes.txt`

Options utiles de `cat`

-n : numérote les lignes.
-E : affiche le caractère « fin de ligne » ($).
-b : numérote uniquement les lignes non vides.

Commande : `tree`

Définition / Rôle : Affiche l'arborescence d'un répertoire sous forme d'arbre.
Syntaxe : `tree [répertoire]`
Exemple : `tree /home/user`

2.2 Commandes de manipulation des fichiers et répertoires

Commande : `touch`

Définition / Rôle : Crée un fichier vide ou met à jour la date de modification.
Syntaxe : `touch fichier`
Exemple : `touch notes.txt`

Commande : `mkdir`

Définition / Rôle : Crée un nouveau répertoire.
Syntaxe : `mkdir [options] répertoire`
Exemple : `mkdir Documents/Cours`

Commande : `cp`

Définition / Rôle : Copie un fichier ou un répertoire.
Syntaxe : `cp [options] source destination`
Exemple : `cp fichier.txt sauvegarde.txt`

Options utiles de `cp`

-r : copie récursive des répertoires.
-i : demande confirmation avant d'écraser.
-v : affiche les actions effectuées.

Commande : `mv`

Définition / Rôle : Déplace ou renomme fichiers et répertoires.
Syntaxe : `mv source destination`
Exemple : `mv ancien.txt nouveau.txt`

Commande : `rm`

Définition / Rôle : Supprime des fichiers ou répertoires.
Syntaxe : `rm [options] fichier`
Exemple : `rm notes.txt`

Options utiles de `rm`

-r : suppression récursive des répertoires.
-i : demande une confirmation.
-f : force la suppression sans confirmation.

3 Analyse détaillée du résultat de `ls -l`

La commande `ls -l` permet d'afficher des informations complètes sur les fichiers d'un répertoire. Considérons l'exemple suivant :

```
user@linux:~$ ls -l -rw-r--r-- 1 user user 2048 Nov  4 14:22 rapport.txt
```

Chaque colonne du résultat correspond à une métadonnée du fichier, fournie par le système de fichiers Linux. Nous détaillons ci-dessous la signification de chaque élément.

3.1 1. Le type du fichier et les permissions

La première colonne est `-rw-r-r-`. Elle contient 10 caractères :

```
[1][2 3 4][5 6 7][8 9 10]
```

— **1er caractère : type du fichier** Le type de fichier sert justement à indiquer la catégorie à laquelle appartient un élément du système, afin que Linux sache comment le gérer et comment l'utilisateur doit l'interpréter.
  — `-` : fichier ordinaire
  — `d` : répertoire
  — `l` : lien symbolique
  — `b`, `c` : périphériques bloc / caractère
  — `s` : socket
  — `p` : tube nommé (FIFO)
— **Les 9 caractères suivants : permissions**
  — **rw-** : permissions du propriétaire (user)
  — **r–** : permissions du groupe
  — **r–** : permissions des autres utilisateurs (others)

1.1. Modes symbolique et octal des permissions

Les permissions peuvent être exprimées selon deux modes :

- **Mode symbolique**
  - `r` = lecture
  - `w` = écriture
  - `x` = exécution
  - `-` = absence de permission

  Exemple : `rw-r-r-`

- **Mode octal** Chaque permission est un bit. On associe : `r=4`, `w=2`, `x=1`.

Catégorie	r	w	x
Propriétaire	4	2	0
Groupe	4	0	0
Autres	4	0	0

Ainsi :

$$rw - r - - r - - = 640 + 40 + 4 = \mathbf{644}$$

1.2. Valeurs par défaut et masque `umask`

Lorsqu'un fichier ou un répertoire est créé sous Linux, il reçoit des permissions initiales définies par le système. Ces permissions combinent :

- **Permissions maximales possibles (permissions par défaut)** : le système attribue le droit maximal que le fichier ou le répertoire pourrait avoir ;
- **Masque `umask`** : un masque qui retire certains droits pour des raisons de sécurité.

La formule générale est :

$$\text{permissions finales} = \text{permissions par défaut} - \text{umask}$$

1. Origine des permissions par défaut

- **Fichiers :** 666 (lecture et écriture pour utilisateur, groupe et autres, pas d'exécution).
- **Répertoires :** 777 (lecture, écriture et exécution pour tous ; l'exécution permet de traverser le répertoire).

Ces valeurs représentent le *maximum théorique* que Linux peut attribuer automatiquement, garantissant sécurité et fonctionnalité.

2. Le masque `umask`

- Le `umask` (**User Mask**) retire certains droits par défaut.
- Chaque bit à 1 dans le `umask` supprime le droit correspondant, chaque bit à 0 laisse le droit actif.

Exemple : `umask = 022` → le groupe et les autres perdent le droit d'écriture.

3. Exemple de calcul

— Création d'un fichier :

$$666 - 022 = 644 \quad \text{(user : rw-, group : r–, others : r–)}$$

— Création d'un répertoire :

$$777 - 022 = 755 \quad \text{(user : rwx, group : r-x, others : r-x)}$$

Ainsi un fichier reçoit :

$$666 - 022 = 644$$

3.2 Nombre de liens (hard links)

Lorsque l'on liste un fichier avec `ls -l`, le deuxième champ affiche le nombre de liens physiques (hard links) associés au fichier. Par exemple :

```
-rw-r--r--  1 user ...
```

Ici, le `1` indique que le fichier possède un seul lien physique.

Lien physique (hard link)

— Un **hard link** est un pointeur direct vers l'inode d'un fichier, c'est-à-dire sa structure de stockage réelle sur le disque.

— Plusieurs noms de fichiers peuvent pointer vers le même inode, permettant ainsi d'accéder au même contenu via des chemins différents.

— La suppression d'un lien ne supprime pas le contenu tant qu'au moins un autre lien existe.

Exemple

copie.txt –> inode 12345 <– rapport.txt

Lien symbolique (symlink)

— Un **symlink** (lien symbolique) est un fichier spécial qui contient le chemin vers un autre fichier ou répertoire.

— Il possède son propre inode distinct de celui du fichier cible.

— Si le fichier cible est supprimé, le symlink devient "cassé" (dangling link).

— Les symlinks peuvent pointer vers des fichiers situés sur d'autres systèmes de fichiers, contrairement aux hard links.

Exemple

rapport.txt <– contenu réel
rapport_link –> rapport.txt (symlink pointant vers rapport.txt)

3.3 Propriétaire et groupe

Dans l'exemple affiché par `ls -l` :

`user user`

— Le **1er** `user` représente le *propriétaire* du fichier, c'est-à-dire l'utilisateur qui possède le fichier et qui a des droits spécifiques sur celui-ci (lecture, écriture, exécution selon les permissions).

— Le **2e** `user` représente le *groupe* auquel appartient le fichier. Chaque utilisateur peut appartenir à un ou plusieurs groupes.

Pourquoi la notion de groupe ? La notion de groupe permet de gérer les permissions de manière collective. Au lieu de définir des droits individuels pour chaque utilisateur, Linux attribue des droits à un groupe, et tous les membres de ce groupe héritent automatiquement des permissions associées.

— Cela simplifie la gestion des accès dans des environnements multi-utilisateurs.

— Cela permet de partager des fichiers ou des répertoires avec un ensemble d'utilisateurs définis sans modifier individuellement chaque permission.

— Le système distingue ainsi trois catégories d'utilisateurs pour chaque fichier : *propriétaire*, *groupe* et *autres*.

3.4 Taille du fichier

`2048`

Il s'agit de la taille réelle du fichier en **octets**. Pour un répertoire, `ls -l` affiche la taille du contenu descriptif (liste des entrées).

3.5 Date et heure de dernière modification

`Nov  4 14:22`

Linux affiche :

— mois abrégé ;

— jour du mois ;

— heure (ou année si le fichier est ancien).

3.6 Nom du fichier

`rapport.txt`

C'est simplement le nom stocké dans l'entrée du répertoire.

3.7 Modifier les permissions : `chmod`

La commande `chmod` permet de modifier les permissions d'un fichier ou d'un répertoire. Elle peut être utilisée en deux modes principaux :

— **Mode symbolique** : modification des permissions en utilisant les lettres `u` (user/propriétaire), `g` (groupe), `o` (others/autres) et `a` (all/tous).

Exemple

`chmod u+r fichier` ajoute le droit de lecture pour le propriétaire.

— **Mode octal** : représentation des permissions par des chiffres de 0 à 7 correspondant aux droits `r=4, w=2, x=1`.

Exemple

`chmod 755 script.sh` attribue `rwx` au propriétaire et `rx` au groupe et aux autres.

Bits spéciaux

Linux propose également des bits spéciaux qui modifient le comportement de l'exécution des fichiers et l'accès aux répertoires :

— **Setuid (s)** : lorsqu'il est défini sur un fichier exécutable, le programme s'exécute avec les privilèges du propriétaire du fichier, et non de l'utilisateur courant.

— **Setgid (s)** : pour un fichier, le programme s'exécute avec les droits du groupe du fichier. Pour un répertoire, tous les fichiers créés héritent du groupe du répertoire.

— **Sticky bit (t)** : appliqué à un répertoire, il empêche les utilisateurs de supprimer ou renommer les fichiers appartenant à d'autres utilisateurs dans ce répertoire (utile pour `/tmp`).

Syntaxe et exemples pratiques

Commande : `chmod`

Définition / Rôle : Modifier les permissions d'un fichier ou répertoire.
Syntaxe :

— Mode symbolique : `chmod [ugoa][+-=][rwx] fichier`

— Mode octal : `chmod [0-7][0-7][0-7] fichier`

— Avec bits spéciaux : `chmod u+s fichier`, `chmod g+s dossier`, `chmod +t dossier`

Exemple :

— `chmod u+x script.sh` : rend `script.sh` exécutable par le propriétaire.

— `chmod 644 rapport.txt` : lecture/écriture pour le propriétaire, lecture seule pour les autres.

— `chmod g+s projet` : les fichiers créés dans `projet` héritent du groupe du répertoire.

4 Commandes avancées et expressions régulières

Les expressions régulières (regex) permettent de rechercher, filtrer et manipuler du texte de manière très flexible dans Linux. Elles constituent un outil puissant pour extraire ou modifier des informations dans des fichiers et flux de données.

Ces expressions sont utilisées par plusieurs commandes essentielles, telles que `grep`, `sed`, `awk`, `find`, `head`, `tail`, `less`, `more`, `wc`, `sort`, `cut`, `uniq`, etc. Nous allons détailler et illustrer l'utilisation de ces commandes juste après, afin de montrer comment les expressions régulières permettent de manipuler efficacement les fichiers et les données sous Linux.

4.1 Récapitulatif des symboles regex

Le tableau ci-dessous présente les principaux métacaractères et constructions d'expressions régulières que vous rencontrerez lors de l'utilisation des commandes Linux :

Motif	**Signification / Utilisation**
.	Représente n'importe quel caractère sauf le saut de ligne.
^	Correspond au début de la ligne.
$	Correspond à la fin de la ligne.
*	Correspond à zéro ou plusieurs occurrences du caractère ou motif précédent.
+	Correspond à une ou plusieurs occurrences du caractère ou motif précédent.
?	Correspond à zéro ou une occurrence du caractère ou motif précédent (optionnel).
[...]	Classe de caractères : correspond à un caractère parmi ceux listés.
[^...]	Classe négative : correspond à un caractère qui n'est pas dans la liste.
{n}	Correspond exactement à n occurrences du motif précédent.
{n,m}	Correspond entre n et m occurrences du motif précédent.
|	Opérateur logique OU entre motifs.
(...)	Groupe de caractères ou motif, utilisé pour l'extraction ou la capture.
\d	Correspond à un chiffre (équivalent à [0-9]).
\D	Correspond à un caractère non numérique.
\s	Correspond à un espace blanc (espace, tabulation, saut de ligne).
\S	Correspond à un caractère non blanc.
\w	Correspond à un caractère alphanumérique ou un underscore (a-z, A-Z, 0-9, _).
\W	Correspond à un caractère non alphanumérique.
\b	Correspond à une frontière de mot.
\B	Correspond à une position qui n'est pas une frontière de mot.

TABLE III.1 – Principaux symboles et motifs des expressions régulières (regex) en Linux

4.2 Commandes utilisant les expressions régulières

Les commandes suivantes exploitent les expressions régulières pour rechercher, filtrer et manipuler du texte.

Commande : `grep`

Définition / Rôle : Recherche dans un fichier ou flux des lignes correspondant à une expression régulière.
Syntaxe : `grep [options] "pattern" fichier(s)`
Exemple : `grep "erreur" /var/log/syslog`

Commande : `sed`

Définition / Rôle : Éditeur de flux non interactif, permet de modifier du texte selon des motifs regex.
Syntaxe : `sed [options] 'commande' fichier`
Exemple : `sed 's/ancien/nouveau/g' fichier.txt`

Commande : `awk`

Définition / Rôle : Langage de traitement de texte et extraction de données basé sur des motifs et champs.
Syntaxe : `awk 'pattern action' fichier`
Exemple : `awk 'print $1' fichier.txt`

Commande : `find`

Définition / Rôle : Recherche des fichiers ou répertoires selon différents critères, pouvant inclure des motifs regex.
Syntaxe : `find [chemin] [options] [expression]`
Exemple : `find /home/user -name "*.txt"`

Commande : `head`

Définition / Rôle : Affiche les premières lignes d'un fichier ou d'un flux.
Syntaxe : `head [options] fichier`
Exemple : `head -n 10 fichier.txt`

Commande : `tail`

Définition / Rôle : Affiche les dernières lignes d'un fichier ou flux, avec possibilité de suivi en temps réel.
Syntaxe : `tail [options] fichier`
Exemple : `tail -f /var/log/syslog`

Commande : `less`

Définition / Rôle : Affiche le contenu d'un fichier page par page avec navigation interactive.
Syntaxe : `less fichier`
Exemple : `less fichier.txt`

Commande : `more`

Définition / Rôle : Affiche le contenu d'un fichier page par page, moins interactif que `less`.
Syntaxe : `more fichier`
Exemple : `more fichier.txt`

Commande : `wc`

Définition / Rôle : Compte le nombre de lignes, mots et caractères dans un fichier ou flux.
Syntaxe : `wc [options] fichier`
Exemple : `wc -l fichier.txt`

Commande : `sort`

Définition / Rôle : Trie les lignes d'un fichier ou flux selon un ordre donné.
Syntaxe : `sort [options] fichier`
Exemple : `sort -r fichier.txt`

Commande : `cut`

Définition / Rôle : Extrait des sections de texte selon des délimiteurs ou positions de caractères.
Syntaxe : `cut [options] fichier`
Exemple : `cut -d ":" -f1 /etc/passwd`

Commande : `uniq`

Définition / Rôle : Supprime les doublons consécutifs dans un fichier ou flux.
Syntaxe : `uniq [options] fichier`
Exemple : `sort fichier.txt | uniq`

5 Archivage et compression

La gestion des fichiers sous Linux ne se limite pas à la création et à la manipulation : il est également essentiel de savoir archiver et compresser les données pour faciliter le stockage, la sauvegarde et le transfert. L'archivage permet de regrouper plusieurs fichiers et répertoires dans un seul fichier, tandis que la compression réduit la taille des fichiers pour économiser de l'espace ou accélérer les transferts.

5.1 Commandes principales d'archivage

Commande : `tar`

Définition / Rôle : Permet de créer, extraire ou lister des archives de fichiers (souvent avec compression).
Syntaxe : `tar [options] archive.tar [fichiers_ou_répertoires]`
Exemple : `tar -cvf archive.tar dossier/` (crée une archive non compressée)

Commande : `tar` avec compression gzip

Définition / Rôle : Archive et compresse en utilisant gzip pour réduire la taille du fichier.
Syntaxe : `tar -czvf archive.tar.gz dossier/`
Exemple : `tar -czvf backup.tar.gz /home/user/Documents`

Commande : `tar` avec compression bzip2

Définition / Rôle : Archive et compresse en utilisant bzip2, plus efficace pour certains fichiers que gzip.
Syntaxe : `tar -cjvf archive.tar.bz2 dossier/`
Exemple : `tar -cjvf backup.tar.bz2 /home/user/Documents`

Commande : `tar` extraction

Définition / Rôle : Permet d'extraire une archive tar ou tar compressée.
Syntaxe : `tar -xvf archive.tar`
Exemple : `tar -xzvf archive.tar.gz`

5.2 Commandes de compression directe

Commande : `gzip`

Définition / Rôle : Compresse un fichier pour réduire sa taille.
Syntaxe : `gzip fichier`
Exemple : `gzip document.txt` (crée document.txt.gz)

Commande : `gunzip`

Définition / Rôle : Décompresse un fichier gzip.
Syntaxe : `gunzip fichier.gz`
Exemple : `gunzip document.txt.gz`

Commande : `bzip2`

Définition / Rôle : Compresse un fichier en utilisant l'algorithme bzip2 pour un meilleur taux de compression.
Syntaxe : `bzip2 fichier`
Exemple : `bzip2 document.txt` (crée document.txt.bz2)

Commande : `bunzip2`

Définition / Rôle : Décompresse un fichier bzip2.
Syntaxe : `bunzip2 fichier.bz2`
Exemple : `bunzip2 document.txt.bz2`

Conclusion

Ce chapitre a présenté les fondements de l'utilisation de Linux : le rôle du shell, l'organisation du système de fichiers, les permissions et les sessions. Nous avons exploré les commandes de base et avancées, ainsi que les expressions régulières, montrant comment naviguer, manipuler et traiter efficacement les données. Enfin, l'archivage et la compression ont été abordés pour gérer et stocker les fichiers de manière optimale.

Chapitre IV

Programmation Shell

Maintenant que nous avons couvert les bases du système Linux et de la ligne de commande, il est temps de commencer à coder. Ce chapitre aborde les notions fondamentales pour écrire des scripts shell. Vous devez connaître ces concepts de base avant de pouvoir créer vos propres scripts shell.

Définition

La programmation en shell, ou scripting shell, désigne l'écriture de scripts constitués de commandes exécutées par un interpréteur de commandes (comme Bash, Zsh ou PowerShell). Elle permet d'automatiser des tâches, de gérer des fichiers et des processus, d'enchaîner des opérations complexes

1 Utiliser plusieurs commandes

L'essentiel des scripts shell réside dans la possibilité d'exécuter plusieurs commandes et de traiter les résultats de chacune, voire même de transmettre la sortie d'une commande à une autre. Le shell vous permet de chaîner des commandes en une seule étape.

1. **En utilisant le ;** Si vous souhaitez exécuter deux commandes successivement, vous pouvez les saisir sur la même ligne d'invite, séparées par un point-virgule.

   Exemple

   ```
   $ date ; who
   Mon Feb 21 15 :36 :09 EST 2014
   Christine tty2 2014-02-21 15 :26
   Samantha tty3 2014-02-21 15 :26
   Timothy tty1 2014-02-21 15 :26
   user tty7 2014-02-19 14 :03
   ```

2. **Exécution conditionnelle (`&&` et `||`)** La commande `&&` permet d'exécuter la commande suivante uniquement si la précédente a réussi, tandis que `||` exécute la commande suivante uniquement si la précédente a échoué.

Exemple

```
$ mkdir nouveau_dossier && echo "Dossier créé avec succès"
```

Exemple

```
$ rm fichier_inexistant || echo "Le fichier n'existe pas"
```

3. **Pipelines (`|`)** permet de passer la sortie d'une commande comme entrée de la commande suivante. Cela permet de chaîner plusieurs commandes et de traiter leurs résultats de manière successive.

Exemple

```
$ ls -l | grep "fichier.txt"
```

Exemple

```
$ cat fichier.txt | sort | uniq
```

4. **Exécution en arrière-plan (`&`)** Le caractère `&` permet d'exécuter une commande en arrière-plan, c'est-à-dire que le shell ne bloque pas et continue à accepter d'autres commandes.

Exemple

```
$ long_processus &
```

5. **Groupement de commandes (`{ }`)** Les accolades `{ }` permettent de regrouper plusieurs commandes pour les exécuter comme un seul bloc. Les commandes à l'intérieur doivent être séparées par des points-virgules.

Exemple

```
$ { echo "Début"; date; echo "Fin"; }
```

Grâce à ces techniques, vous pouvez enchaîner autant de commandes que vous le souhaitez, jusqu'à atteindre le nombre maximal de 255 caractères autorisés dans une ligne de commande. Pour des scripts plus longs ou plus complexes, il sera nécessaire d'écrire vos commandes dans un fichier shell (`.sh`) que vous pourrez ensuite exécuter.

2 Créer un fichier script

Pour placer des commandes shell dans un fichier texte, vous devez d'abord utiliser un éditeur de texte (exemple : gedit , nano , vim) pour créer le fichier, puis y saisir les commandes.

Lors de la création d'un fichier de script shell, vous devez indiquer le shell utilisé (voir chapitre II section 2 dans la première ligne du fichier. Le format est le suivant :

```
#!/bin/bash
```

Dans une ligne normale de script shell, le caractère dièse (#) sert à écrire un commentaire. Une ligne de commentaire n'est pas traitée par le shell. Cependant, la première ligne du script est un cas particulier : le dièse suivi d'un point d'exclamation (#!) indique au shell quel interpréteur utiliser pour exécuter le script, même si vous êtes actuellement dans un autre shell.

Après cette ligne, les commandes sont saisies une par ligne, chacune suivie d'un retour chariot. Comme mentionné, il est possible d'ajouter des commentaires en utilisant le caractère dièse.

Un exemple de script :

```
#!/bin/bash  # Ce script affiche la date et les utilisateurs connectés
date who
```

Et c'est tout ! Vous pouvez également utiliser un point-virgule pour placer plusieurs commandes sur la même ligne, mais dans un script shell, il est courant de mettre chaque commande sur une ligne séparée. Le shell traite les commandes dans l'ordre où elles apparaissent dans le fichier.

Pour exécuter un script, le fichier doit être rendu exécutable en ajoutant le droit d'exécution : il suffit d'utiliser la commande chmod +x script.sh, puis de le lancer avec ./script.sh.

3 Les Variables dans le Shell

L'utilisation des variables constitue un élément fondamental dans la rédaction de scripts Shell. Elles permettent de stocker, manipuler et transmettre des informations au sein d'un programme. La maîtrise des règles de définition et de gestion des variables est donc essentielle pour écrire des scripts fiables et maintenables.

3.1 Syntaxe et règles de nomination

La définition d'une variable dans un environnement Shell obéit à un ensemble de conventions strictes permettant d'assurer la cohérence et la lisibilité du code. De manière générale, un identificateur de variable doit respecter les règles suivantes :

— Il est composé uniquement de lettres, de chiffres et du caractère de soulignement (_).

— Il ne peut pas débuter par un chiffre, afin d'éviter toute ambiguïté avec des valeurs numériques.

— Sa longueur ne doit pas dépasser une vingtaine de caractères, afin de préserver la clarté du script.

— Il est sensible à la casse : une variable en majuscules est distincte de sa version en minuscules.

— Il ne peut pas correspondre au nom d'une commande existante du système, afin d'éviter les conflits d'interprétation lors de l'exécution.

3.2 Deux types de variables

Dans le Shell, les variables peuvent être classées en deux catégories distinctes selon leur portée et leur mode d'utilisation. Cette distinction est essentielle pour comprendre la manière dont les données circulent entre un script, le Shell et les processus qu'il génère.

— **Variables d'environnement** : Il s'agit de variables accessibles par le Shell ainsi que par l'ensemble des sous-processus (scripts, commandes ou programmes) qu'il lance. Elles jouent un rôle central dans la configuration du système et dans la transmission d'informations entre processus.

— **Variables utilisateur** : Ces variables sont définies par l'utilisateur ou au sein d'un script. Elles sont locales au Shell ou au script dans lequel elles apparaissent et servent principalement à stocker et manipuler des données nécessaires à l'exécution du programme.

3.3 Variables d'environnement

Définition

Une variable d'environnement est une variable définie dans le shell et automatiquement accessible par tous les sous-processus, tels que les sous-shells, les scripts ou les programmes exécutés depuis ce shell. Elle reste donc accessible au-delà du processus dans lequel elle a été créée.

À la différence d'une variable normale, qui n'est visible que dans le shell ou le script courant, une variable d'environnement est propagée aux processus enfants.

Commandes importantes

Il est essentiel de connaître les commandes fondamentales qui permettent de les créer, les exporter, les afficher ou les supprimer.

Commande : `export`

Définition / Rôle : Rend une variable accessible aux sous-processus (variable globale).
Syntaxe : `export NOM`
Exemple : `export VAR`

Commande : `declare -x`

Définition / Rôle : Déclare et exporte une variable d'environnement.
Syntaxe : `declare -x NOM`
Exemple : `declare -x VAR=fr_FR.UTF-8`

Commande : `printenv`

Définition / Rôle : Affiche l'ensemble des variables d'environnement.
Syntaxe : `printenv`
Exemple : `printenv`

Commande : `unset`

Définition / Rôle : Supprime une variable d'environnement ou une variable locale.
Syntaxe : `unset NOM`
Exemple : `unset VAR`

Variables d'environnement courantes

Le système Unix met à disposition un ensemble de variables d'environnement préconfigurées, indispensables au fonctionnement du shell et des programmes. Ces variables fournissent des informations essentielles sur l'utilisateur, son environnement d'exécution, ainsi que les chemins utilisés pour localiser les commandes. Elles sont automatiquement définies lors de l'ouverture d'une session et sont héritées par tous les processus lancés dans le shell. Le tableau suivant présente les variables d'environnement les plus courantes et leur rôle dans le système.

Variable	Description
`PATH`	Répertoires où le système cherche les commandes
`HOME`	Répertoire personnel de l'utilisateur
`USER`	Nom de l'utilisateur courant
`SHELL`	Shell utilisé
`PWD`	Répertoire courant

Limites

— La variable exportée est visible dans les sous-processus, mais **pas** dans le shell parent si elle y est modifiée.

— Les programmes déjà lancés avant l'export ne reçoivent pas la variable.

— Pour qu'une variable soit disponible dans tous les futurs terminaux, elle doit être ajoutée dans : `~/.bashrc` ou `~/.bash_profile`.

Exemple

```
export VAR=10
( VAR=20 )
echo $VAR    # Affiche 10
```

Exemple d'exécution dans un sous-shell :

```
bash -c 'echo "Depuis le sous-shell : $MON_VAR"'
```

3.4 Variables utilisateur

Un script shell permet de définir et d'utiliser ses propres variables. Ces variables existent **seulement pendant l'exécution du script**.

Règles

— Affectation : `variable=valeur` (sans espaces).

— Le type est déterminé automatiquement par le shell.

Exemple

```
#!/bin/bash
# testing variables
days=10
guest="Katie"
echo "$guest checked in $days days ago"

days=5
guest="Jessica"
echo "$guest checked in $days days ago"
```

Important : Pour lire la valeur d'une variable → `$variable`
Pour affecter → `variable=valeur` (sans $)

3.5 Attributs des variables

Les variables Shell peuvent se voir associer des attributs qui permettent de contrôler leur comportement : type (entier), modification autorisée ou non, etc. Ces attributs sont définis principalement à l'aide de la commande `declare`. Ils permettent d'éviter des erreurs dans les scripts et de garantir un fonctionnement plus prévisible.

1. **Rendre une variable en lecture seule**
   Il est parfois nécessaire de protéger une variable afin qu'elle ne puisse plus être modifiée une fois définie. Pour cela, le Shell propose l'attribut `readonly`.

   — `readonly NOM` : rend la variable impossible à modifier.
   — `declare -r NOM` : équivalent avec la commande `declare`.

   Exemple

   ```
   #!/bin/bash
   univ=usthb
   echo $univ
   readonly univ
   univ=esi   # Erreur : variable en lecture seule
   ```

2. **Forcer le type d'une variable (attributs de typage)**
   Le Shell permet d'imposer un type particulier à une variable afin de contrôler son comportement. L'attribut le plus courant est `-i`, qui force une variable à être traitée comme un entier. Dans ce cas, Bash réalise automatiquement les opérations arithmétiques sans recourir à `expr` ou `$(( ))`.

   — `declare -i NOM` : déclare une variable entière et évalue automatiquement les expressions numériques.

Exemple

```
#!/bin/bash
declare -i nombre="10 + 5"   # Calcul automatique
echo "Variable entière nombre = $nombre"

declare -r version="1.0"
echo "Variable readonly version = $version"
```

3. **Supprimer une variable**
   Pour réinitialiser complètement une variable, ou libérer un nom, on utilise la commande `unset`. Après suppression, la variable n'existe plus dans l'environnement d'exécution.

Exemple

```
#!/bin/bash
temp="donnée temporaire"
echo "Avant unset : $temp"
unset temp
echo "Après unset : $temp"
```

3.6 Lecture de valeurs avec `read`

La commande `read` permet de récupérer des entrées utilisateur, que ce soit depuis le clavier ou depuis un fichier. Elle est particulièrement utile pour interagir avec l'utilisateur dans un script.

Exemple

```
#!/bin/bash

# Lire un nom et l'afficher
read -p "Entrez votre nom : " nom
echo "Bonjour, $nom !"

# Lire un mot de passe sans l'afficher à l'écran
read -s -p "Entrez votre mot de passe : " mdp
echo
echo "Mot de passe enregistré (non affiché pour sécurité)"
```

Explications :

— `-p` : affiche un message avant de lire l'entrée.

— `-s` : empêche l'affichage de ce qui est tapé (utile pour les mots de passe).

— La valeur saisie est stockée dans la variable indiquée (`nom` ou `mdp`).

3.7 Les tableaux

Les tableaux en Bash permettent de stocker plusieurs valeurs dans une seule variable, accessibles par des indices numériques.

— Un tableau se déclare avec `declare -a`.
— Les valeurs sont accessibles via des indices : `tableau[0]`, `tableau[1]`, etc.
— Les indices n'ont pas besoin d'être consécutifs.

Opérations courantes

Opération	Description
`declare -a tableau`	Déclare un tableau vide
`tableau[i]=valeur`	Ajoute ou modifie l'élément d'indice `i`
`$tableau[i]`	Accède à un élément précis
`$#tableau[@]`	Taille du tableau (nombre d'éléments)
`tableau+=( "élément" )`	Ajoute un élément à la fin du tableau
`unset tableau[i]`	Supprime l'élément d'indice `i`

Exemple

```
#!/bin/bash

declare -a fruits     # Déclaration d'un tableau

fruits[0]="Pomme"
fruits[1]="Banane"
fruits+=( "Orange" )  # Ajout en fin de tableau

echo "Premier fruit : ${fruits[0]}"
echo "Tous les fruits : ${fruits[@]}"
echo "Nombre d'éléments : ${#fruits[@]}"

unset fruits[1]        # Suppression de l'élément d'indice 1
echo "Après suppression : ${fruits[@]}"
```

3.7.1 Boucle sur un tableau

Pour parcourir tous les éléments d'un tableau, on peut utiliser une boucle `for` :

Exemple

```
#!/bin/bash

fruits=("Pomme" "Banane" "Orange")

for element in "${fruits[@]}"; do
    echo $element
done
```

Explications :

— `"$fruits[@]"` : récupère tous les éléments du tableau.

— `for element in ...` : itère sur chaque élément du tableau.

— `do ... done` : délimite le bloc de commandes à exécuter pour chaque élément.

4 Calculs et Expressions dans le Shell

Dans les scripts Shell, il est souvent nécessaire d'effectuer des opérations arithmétiques, des comparaisons ou encore des manipulations de chaînes de caractères. La commande `expr` constitue l'un des outils historiques permettant d'exécuter ce type d'expressions. Nous présentons ci-dessous ses principaux usages de manière pédagogique.

4.1 Opérations avec `expr`

La commande `expr` permet de réaliser :

— des calculs numériques,

— des comparaisons,

— des recherches dans les chaînes de caractères.

1) Opérateurs arithmétiques

Opérateur	Rôle
+	Addition
-	Soustraction
*	Multiplication (nécessite un antislash)
/	Division
%	Modulo

Exemple

```
#!/bin/bash
a=10
b=5
echo "Somme : $(expr $a + $b)"
echo "Différence : $(expr $a - $b)"
echo "Produit : $(expr $a \* $b)"
echo "Division : $(expr $a / $b)"
echo "Modulo : $(expr $a % $b)"
```

2) Opérateurs de comparaison

Opérateur	Rôle
=	Égalité (renvoie 1 si vrai, sinon 0)
!=	Différent
<	Inférieur
>	Supérieur
<=	Inférieur ou égal
>=	Supérieur ou égal

Exemple

```
#!/bin/bash
a=10
b=5
echo "Comparaisons numériques :"
echo "$a = $b : $(expr $a = $b)"
echo "$a != $b : $(expr $a != $b)"
echo "$a > $b : $(expr $a \> $b)"
echo "$a < $b : $(expr $a \< $b)"
```

3) Opérateurs pour les chaînes

Opérateur	Rôle
`index`	Position de la première occurrence d'un caractère
`length`	Longueur de la chaîne
`:`	Extraction ou recherche d'un motif

Exemple

```
#!/bin/bash
texte="Bonjour tout le monde"
char="o"
motif="Bonjour"
echo "Longueur de la chaîne : $(expr length "$texte")"
echo "Première occurrence de '$char' : $(expr index "$texte" "$char")"
echo "Caractères correspondant au motif '$motif' :
$(expr "$texte" : "$motif")"
```

5 Calculs avec l'arithmétique Bash

Bash propose une syntaxe interne pour effectuer des calculs sur des entiers. Cette approche est plus moderne et plus simple que l'utilisation de `expr`.

Méthodes disponibles

— **Expansion arithmétique** : `$(( ... ))`, pour évaluer directement une expression.

— **Commande** `let` : permet d'exécuter des opérations arithmétiques sur des variables.

Exemple

```
#!/bin/bash

a=10
b=5
# Utilisation de l'expansion arithmétique
echo "a + b = $((a + b))"   # Affiche 15
# Utilisation de la commande let
let somme=a+b
echo "somme = $somme"       # Affiche 15
```

Explications :

— `$(( ... ))` : calcule directement l'expression et retourne le résultat.

— `let` : permet d'affecter le résultat d'une expression arithmétique à une variable existante.

— Ces méthodes fonctionnent uniquement avec des entiers.

6 Tests de Conditions dans le Shell

Les tests conditionnels permettent aux scripts Shell de prendre des décisions en fonction de valeurs numériques, de chaînes de caractères ou de propriétés de fichiers. Ils utilisent principalement les commandes `test`, `[ ... ]` et `[[ ... ]]`.

6.1 Tests sur les valeurs numériques

Dans les comparaisons numériques :

— les chaînes sont automatiquement converties en entiers ;

— Bash ne gère que des valeurs numériques **entières** ;

— la syntaxe générale est : `test val1 option val2`.

Opérateurs numériques

Option	Rôle
`-eq`	Égal (equal)
`-ne`	Différent (not equal)
`-lt`	Inférieur (less than)
`-gt`	Supérieur (greater than)
`-le`	Inférieur ou égal
`-ge`	Supérieur ou égal

Exemple : Tests numériques

Exemple

```
#!/bin/bash
a=10
b=5
c=10
test $a -eq $c && echo "$a est égal à $c"
test $a -ne $b && echo "$a est différent de $b"
test $a -gt $b && echo "$a est plus grand que $b"
test $b -lt $a && echo "$b est plus petit que $a"
test $a -ge $c && echo "$a est supérieur ou égal à $c"
test $b -le $a && echo "$b est inférieur ou égal à $a"
```

6.2 Tests sur les chaînes de caractères

Opérateurs

Test	Rôle
`test -z "var"`	Vrai si la chaîne est vide
`test -n "var"`	Vrai si la chaîne n'est pas vide
`test "var" = chaine`	Vrai si les deux chaînes sont identiques
`test "var" != chaine`	Vrai si les deux chaînes sont différentes

Exemple

```
#!/bin/bash

a=""
b="Hello"
c="abc"
d="abc"
e="abc"
f="def"

test -z "$a" && echo "a est vide"
test -n "$b" && echo "b n'est pas vide"
test "$c" = "$d" && echo "c et d sont identiques"
test "$e" != "$f" && echo "e et f sont différentes"
```

6.3 Tests sur les fichiers

Principaux opérateurs

Option	Rôle
`-f`	Fichier normal
`-d`	Répertoire
`-b`	Fichier en mode bloc
`-c`	Fichier en mode caractère
`-p`	Tube nommé (named pipe)
`-r`	Lecture autorisée
`-w`	Écriture autorisée
`-x`	Exécution autorisée
`-s`	Fichier non vide
`-e`	Le fichier existe
`-L`	Lien symbolique
`-u`	Bit SUID positionné
`-g`	Bit SGID positionné

6.4 Syntaxe allégée

— `[ ... ]` est une forme simplifiée de `test`.

— Il faut obligatoirement des espaces autour des crochets.

— `[[ ... ]]` est plus moderne, permet des comparaisons alphabétiques et protège mieux les chaînes.

Exemple

```
#!/bin/bash

# Vide / non vide
[ -z "$a" ] && echo "a est vide"
[ -n "$b" ] && echo "b n'est pas vide"

# Comparaison de chaînes
[ "$c" = "$d" ] && echo "c et d sont identiques"
[ "$e" != "$f" ] && echo "e et f sont différentes"

# Comparaison lexicographique (alphabetique)
[[ "$c" < "$f" ]] && echo "$c vient avant $f"
[[ "$f" > "$c" ]] && echo "$f vient après $c"
```

7 Structures de contrôle

Les structures de contrôle en `bash` permettent d'adapter le comportement d'un script en fonction de conditions, de répéter des actions ou encore de proposer des choix à l'utilisateur. Elles rendent les scripts plus dynamiques, lisibles et puissants. Nous présentons ci-dessous les principales structures : `if`, `for`, `while`, `until`, `select`, `case`, ainsi que les commandes `break` et `continue`.

7.1 Structure `if`

La structure `if` permet d'exécuter des instructions si une condition est vraie.

```
if [ condition ]; then
    commandes
elif [ autre_condition ]; then
    commandes
else
    commandes
fi
```

Exemple

```
#!/bin/bash
a=10

if [ $a -gt 5 ]; then
    echo "a est supérieur à 5"
else
    echo "a est inférieur ou égal à 5"
fi
```

7.2 Boucle `for`

La boucle `for` permet d'exécuter une série de commandes pour chaque élément d'une liste.

```
for variable in liste; do
    commandes
done
```

Exemple

```
for i in 1 2 3 4 5; do
    echo "Iteration : $i"
done
```

7.3 Boucle `while`

La boucle `while` s'exécute tant qu'une condition est vraie.

```
while [ condition ]; do
    commandes
done
```

Exemple

```
i=1
while [ $i -le 5 ]; do
    echo "i = $i"
    i=$((i + 1))
done
```

7.4 Boucle `until`

La boucle `until` s'exécute tant que la condition est **fausse**. Elle s'arrête lorsque la condition devient vraie.

```
until [ condition ]; do
    commandes
done
```

Exemple

```
i=1
until [ $i -gt 5 ]; do
    echo "i = $i"
    i=$((i + 1))
done
```

7.5 Structure `select`

Très utile pour proposer un menu interactif à l'utilisateur.

```
select choix in option1 option2 option3; do
    commandes
done
```

Exemple

```
select fruit in Pomme Orange Banane Quitter; do
    echo "Vous avez choisi : $fruit"
    [ "$fruit" = "Quitter" ] && break
done
```

7.6 Structure `case`

La structure `case` est idéale pour traiter plusieurs cas différents (selon une valeur).

```
case variable in
    motif1) commandes ;;
    motif2) commandes ;;
    *) commandes par défaut ;;
esac
```

Exemple

```
read -p "Entrez une lettre : " l

case $l in
    a) echo "Vous avez tapé a" ;;
    b) echo "Vous avez tapé b" ;;
    *) echo "Autre lettre" ;;
esac
```

7.7 break et continue

— **break** : quitte immédiatement la boucle.

— **continue** : saute à l'itération suivante.

Exemple

```
for i in 1 2 3 4 5; do
    if [ $i -eq 3 ]; then
        continue   # saute le 3
    fi
    if [ $i -eq 5 ]; then
        break      # stop la boucle
    fi
    echo $i
done
```

8 Les Fonctions en Shell

Qu'est-ce qu'une fonction en Shell ? Une fonction en Shell est un ensemble de commandes regroupées sous un nom. Elle s'exécute comme une seule instruction et peut être appelée à tout moment dans un script. En d'autres termes, c'est un mini-programme à l'intérieur de votre script. Les fonctions permettent de :

— Évite la répétition de code

— Code plus lisible

— Maintenance simplifiée

— Modularité

— Réutilisation

— Gestion facilitée des tâches complexes

8.1 Création d'une fonction Shell

Deux syntaxes sont possibles pour définir une fonction en Bash :

— Avec le mot-clé `function` :

Exemple

```
function nom {
    commandes
}
```

— Syntaxe classique (comme dans d'autres langages) :

Exemple

```
nom() {
    commandes
}
```

8.2 Utilisation des fonctions

Exemple

```
#!/bin/bash

function func1 {
    echo "This is an example of a function"
}

count=1
while [ $count -le 5 ]; do
    func1
    count=$(( $count + 1 ))
done
```

Remarques :

— L'appel de la fonction doit apparaître après sa définition.

— Le nom de la fonction doit être unique dans le script.

8.3 La valeur de retour d'une fonction

Par défaut, le code de retour d'une fonction correspond à celui de la dernière commande exécutée. On peut le récupérer avec la variable `$?`.

Exemple

```
#!/bin/bash

func1() {
    echo "Trying to display a non-existent file"
    ls -l badfile
}

func1
echo "The exit status is: $?"
```

Utiliser la commande `return`

Exemple

```
#!/bin/bash

function dbl {
    read -p "Enter a value: " value
    echo "Doubling the value"
    return $(( value * 2 ))
}

dbl
echo "The new value is $?"
```

Remarque : La commande `return` ne peut renvoyer qu'un entier entre 0 et 255. Pour retourner d'autres valeurs ou des chaînes, il faut utiliser `echo` et capturer la sortie.

Capturer la sortie d'une fonction

Exemple

```
#!/bin/bash

function dbl {
    read -p "Enter a value: " value
    echo $(( value * 2 ))
}

result=$(dbl)
echo "The new value is $result"
```

8.4 Passage de paramètres aux fonctions

Les fonctions peuvent recevoir des arguments accessibles via les variables spéciales :

— $1, $2, ... $9 : paramètres passés à la fonction.

— $# : nombre total d'arguments.

— $0 : nom de la fonction en cours d'exécution.

Exemple

```
#!/bin/bash

function addem {
    if [ $# -eq 0 ] || [ $# -gt 2 ]; then
        echo -1
    elif [ $# -eq 1 ]; then
        echo $(( $1 + $1 ))
    else
        echo $(( $1 + $2 ))
    fi
}

echo -n "Adding 10 and 15: "
value=$(addem 10 15)
echo $value
```

8.5 Redéfinition et réorganisation des paramètres

— `set` : permet de redéfinir le contenu des paramètres positionnels.

— `shift` : décale les paramètres vers la gauche.

Exemple

```
#!/bin/bash

set 10 20 30 40
echo "Avant shift:"
echo "Nombre de paramètres: $#"
echo "Deux premiers: $1, $2"
shift
echo "Après shift:"
echo "Nombre de paramètres: $#"
echo "Deux premiers: $1, $2"
```

8.6 Variables locales et globales

— Les variables globales sont valables partout dans le script.

— Le mot-clé `local` limite la portée d'une variable à l'intérieur de la fonction.

Chapitre V

Administration système

Définition

L'administration système désigne l'ensemble des activités visant à gérer et maintenir un système informatique afin d'assurer son fonctionnement de manière efficace, sécurisée et fiable. Elle comprend notamment la configuration, la surveillance et la maintenance des ressources matérielles et logicielles.

Les principales fonctions de l'administration système incluent :

- **Gestion des utilisateurs et des droits d'accès :** Création, modification et suppression des comptes utilisateurs ainsi que l'attribution de permissions adaptées pour garantir la sécurité et la confidentialité des données.
- **Gestion du stockage :** Allocation, organisation et optimisation des espaces de stockage pour assurer la disponibilité et la performance des données.
- **Gestion des processus et des services :** Supervision et contrôle des processus en cours d'exécution et des services système afin d'assurer la continuité et la stabilité des applications.
- **Surveillance et maintenance :** Contrôle régulier des performances et de l'intégrité du système, détection des anomalies et application des correctifs nécessaires.
- **Sécurité du système :** Mise en place de mesures de protection contre les accès non autorisés, les logiciels malveillants et les vulnérabilités potentielles.

1 Répertoires Système

Les systèmes de fichiers des systèmes d'exploitation de type Unix/Linux sont organisés selon une hiérarchie précise, où chaque répertoire a une fonction spécifique. Les principaux répertoires sont décrits ci-dessous :

- **/bin :** Contient les commandes essentielles du système accessibles à tous les utilisateurs, nécessaires pour le fonctionnement de base du système.
- **/boot :** Contient les fichiers requis pour le démarrage du système, notamment le noyau (*kernel*) et les fichiers de configuration du chargeur de démarrage.
- **/dev :** Contient les fichiers spéciaux représentant les périphériques matériels (disques, claviers, imprimantes, etc.), permettant l'interaction avec le matériel via des interfaces standards.

— /**etc :** Contient la majorité des fichiers de configuration du système et des services installés.

— /**home :** Contient les répertoires personnels des utilisateurs, où sont stockées leurs données et configurations spécifiques.

— /**lib :** Contient les bibliothèques partagées essentielles au fonctionnement des programmes et du système.

— /**lost+found :** Contient les fragments de fichiers récupérés par l'outil `fsck` après une vérification du système de fichiers.

— /**mnt,** /**media :** Contiennent les points de montage temporaires utilisés pour monter des systèmes de fichiers externes ou supplémentaires.

— /**proc :** Contient des informations sur l'état du système et les processus en cours d'exécution, fournissant une interface virtuelle vers le noyau.

— /**root :** Répertoire personnel de l'administrateur système (*root*).

— /**sbin :** Contient les commandes essentielles réservées à l'administrateur système pour la maintenance et la configuration.

— /**tmp :** Contient les fichiers temporaires créés par les utilisateurs et les applications, souvent sur une partition dédiée.

— /**var :** Contient les données susceptibles d'être fréquemment modifiées, telles que les bases de données et fichiers temporaires d'application.

— /**var**/**log :** Contient les journaux du système et des services, permettant le suivi des événements et le diagnostic.

— /**var**/**spool :** Contient les spouleurs, notamment pour l'impression et le courrier électronique, en attente de traitement.

2 Gestion des Utilisateurs et Groupes

2.1 Fichiers liées à la gestion des utilisateurs et groupes

Dans les systèmes Unix/Linux, les informations relatives aux utilisateurs et aux groupes sont stockées dans des fichiers spécifiques situés dans le répertoire `/etc`. Ces fichiers sont essentiels pour la gestion des comptes, des mots de passe et des permissions.

— /**etc**/**passwd :** Contient les informations de base sur tous les comptes utilisateurs. Chaque ligne correspond à un utilisateur et suit le format :

```
nom_utilisateur:x:UID:GID:commentaire:répertoire_home:shell
```

  — `nom_utilisateur` : nom du compte.
  — `x` : indique que le mot de passe est stocké dans `/etc/shadow`.
  — `UID` : identifiant unique de l'utilisateur.
  — `GID` : identifiant du groupe principal.
  — `commentaire` : informations supplémentaires, souvent le nom complet.
  — `répertoire_home` : répertoire personnel de l'utilisateur.
  — `shell` : interpréteur de commandes par défaut.

— **/etc/shadow :** Contient les informations de mot de passe chiffré et les politiques de sécurité des comptes utilisateurs. L'accès est restreint aux utilisateurs privilégiés pour des raisons de sécurité. Chaque ligne suit le format :

```
nom_utilisateur:mot_de_passe_chiffré:dernière_modification:min:max:avertissem
```

  — `mot_de_passe_chiffré` : mot de passe crypté.
  — `dernière_modification` : date de dernier changement du mot de passe (en jours depuis le 1er janvier 1970).
  — `min`, `max` : durée minimale et maximale entre changements de mot de passe.
  — `avertissement` : nombre de jours avant expiration pour avertir l'utilisateur.
  — `expiration` : date d'expiration du compte.
  — `inactif` : nombre de jours après expiration avant désactivation du compte.

— **/etc/group :** Contient les informations sur les groupes du système. Chaque ligne correspond à un groupe et suit le format :

```
nom_groupe:x:GID:liste_utilisateurs
```

  — `nom_groupe` : nom du groupe.
  — `x` : indique que le mot de passe du groupe est stocké dans `/etc/gshadow` (rarement utilisé).
  — `GID` : identifiant unique du groupe.
  — `liste_utilisateurs` : utilisateurs membres du groupe, séparés par des virgules.

2.2 Création d'un Utilisateur

La création et la gestion des comptes utilisateurs sont des tâches essentielles de l'administration système. Plusieurs méthodes permettent de créer un utilisateur sous Unix/Linux, chacune adaptée à différents besoins.

2.2.1 Méthode 1 : Utilisation de la commande `useradd`

La commande `useradd` permet de créer un nouvel utilisateur avec des options pour définir son répertoire personnel, son groupe et son shell par défaut.

Exemple

```
sudo useradd -m -s /bin/bash -G groupe_utilisateur nom_utilisateur
```

— `-m` : crée automatiquement le répertoire personnel.
— `-s` : définit le shell par défaut.
— `-G` : ajoute l'utilisateur à un ou plusieurs groupes supplémentaires.

Après la création, il est souvent nécessaire de définir un mot de passe à l'aide de la commande `passwd` :

```
sudo passwd nom_utilisateur
```

2.2.2 Méthode 2 : Utilisation de la commande `adduser`

`adduser` est un script interactif plus convivial qui guide l'administrateur dans la création du compte utilisateur.

Exemple

```
sudo adduser nom_utilisateur
```

Cette commande demande successivement le mot de passe, le nom complet, le numéro de téléphone et d'autres informations facultatives. Elle crée automatiquement le répertoire personnel et définit les permissions appropriées.

2.2.3 Méthode 3 : Modification manuelle des fichiers système

Pour des cas avancés ou spécifiques, un administrateur peut directement modifier les fichiers `/etc/passwd`, `/etc/shadow` et `/etc/group` pour ajouter un utilisateur. Cette méthode est rarement utilisée en pratique car elle comporte des risques d'erreurs critiques, mais elle permet un contrôle complet sur les champs et paramètres de l'utilisateur.

Conclusion

Les deux méthodes principales pour créer un utilisateur restent `useradd` et `adduser`, qui automatisent les tâches essentielles telles que la création du répertoire personnel et l'affectation aux groupes. La modification directe des fichiers système est réservée aux situations exceptionnelles et aux administrateurs expérimentés.

2.3 Modification d'un Utilisateur

La modification des comptes utilisateurs est une tâche fréquente en administration système. Elle permet de mettre à jour des informations telles que le répertoire personnel, le shell par défaut, les groupes ou encore le mot de passe. Plusieurs méthodes sont disponibles pour effectuer ces modifications.

2.3.1 Méthode 1 : Utilisation de la commande `usermod`

La commande `usermod` est l'outil principal pour modifier un utilisateur existant. Quelques options courantes sont :

- `-l nouveau_nom` : change le nom de l'utilisateur.
- `-d nouveau_home -m` : change le répertoire personnel et déplace les fichiers existants.
- `-s nouveau_shell` : change le shell par défaut.
- `-G groupes` : ajoute ou remplace l'utilisateur dans des groupes supplémentaires.
- `-L` / `-U` : verrouille ou déverrouille le compte.

Exemple

```
sudo usermod -d /home/nouveau_home -m -s /bin/zsh -G sudo nom_utilisateur
```

2.3.2 Méthode 2 : Modification du mot de passe avec `passwd`

Le mot de passe d'un utilisateur peut être changé indépendamment de `usermod` en utilisant la commande `passwd` :

```
sudo passwd nom_utilisateur
```

Le système demandera le nouveau mot de passe et mettra à jour le fichier `/etc/shadow`.

2.3.3 Méthode 3 : Modification directe des fichiers système

Pour des cas avancés, il est possible de modifier directement les fichiers `/etc/passwd`, `/etc/shadow` et `/etc/group`. Cette méthode nécessite une grande prudence, car une erreur peut rendre le compte inutilisable ou créer des problèmes de sécurité. Elle est rarement utilisée en pratique.

Conclusion

La commande `usermod` constitue la méthode standard pour modifier les informations d'un utilisateur, tandis que `passwd` permet de mettre à jour le mot de passe. La modification directe des fichiers système doit être réservée aux administrateurs expérimentés et à des situations exceptionnelles.

Chapitre VI

Introduction à Python pour la Bioinformatique

1 Pourquoi Python pour la bioinformatique ?

Python est un langage de programmation très adapté à la bioinformatique pour plusieurs raisons :

- **Orienté objet :** Python permet la programmation orientée objet, facilitant la modélisation des entités biologiques sous forme de classes et d'objets.
- **Structuration du programme :** Il permet de structurer le code en unités modulaires, facilitant la maintenance et la réutilisation.
- **Facilité des modifications et extensions :** La syntaxe claire et lisible simplifie la compréhension et l'évolution du code.
- **Gestion des exceptions :** Python intègre un système d'exceptions robuste pour gérer les erreurs.
- **Extensible et interfaçable :** Il peut être étendu et interfacer avec d'autres langages comme C ou C++ pour des performances critiques.
- **Disponibilité de bibliothèques :** Python dispose d'un vaste écosystème de bibliothèques scientifiques (`BioPython`, `NumPy`, `Pandas`, `Matplotlib`, etc.) facilitant l'analyse de données biologiques.

2 Types de données en Python

Python propose différents types de données adaptés à la bioinformatique :

- **Entiers :** peuvent être écrits en décimal, hexadécimal (`0x`), binaire (`0b`) ou octal (`0o`).
- **Nombres flottants :** utilisent la notation à virgule flottante et peuvent être exprimés en notation scientifique.
- **Nombres complexes :** représentés sous la forme $a+bj$, où j est la partie imaginaire.
- **Booléens :** valeurs `True` et `False`, correspondant respectivement aux entiers 1 et 0.
- **Chaînes de caractères :** séquences immuables utilisées pour manipuler du texte.

— **Structures de données :** listes, tuples, dictionnaires et ensembles.
— **Valeur spéciale `None` :** représente l'absence de valeur ou le retour par défaut d'une fonction.

Conversion de types :

— `str(arg)` : convertit en chaîne de caractères.
— `int(arg)` : convertit en entier.
— `float(arg)` : convertit en réel.
— `bool(arg)` : retourne `False` pour `None`, zéros, chaînes vides, etc., et `True` sinon.

3 Variables

— Noms valides : lettres (a–z, A–Z), chiffres (0–9), et underscore (_). Doit commencer par une lettre.
— Sensibles à la casse et différents des mots réservés (`if`, `for`, `import`, etc.).
— Affectation avec `=`, affectation multiple possible : `a, b = 5, 10`.
— Opérateurs d'affectation augmentée : `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`.
— Typage dynamique : une variable peut référencer des objets de types différents au cours du programme.
— Variables locales : déclarées dans une fonction. Variables globales : déclarées au niveau du module (mot-clé `global`).

4 Expressions et opérateurs

4.1 Opérateurs numériques

— `**` : puissance.
— `/` : division flottante.
— `//` : division entière.
— `%` : reste de la division entière.
— `+` et `-` : opérateurs unaires ou binaires.

4.2 Précédence des opérateurs

1. Parenthèses
2. Appels
3. Slicings
4. Subscriptions
5. `**` (associativité droite)
6. `+` et `-` unaires
7. `*`, `/`, `//`, `%`
8. `-`, `+`

9. Comparaisons : `==`, `!=`, `<`, `<=`, `>`, `>=`
10. `in`, `not in`
11. `not`
12. `and`
13. `or`

5 Instructions conditionnelles

```
if condition:
    # bloc d'instructions
elif autre_condition:
    # bloc alternatif
else:
    # bloc sinon
```

6 Boucles

6.1 Boucle `while`

```
while condition:
    # instructions
    if condition_arret:
        break
    continue
```

6.2 Boucle `for`

```
for element in sequence:
    # instructions
    if condition_arret:
        break
    continue
```

Instructions de contrôle :

- `break` : interrompt la boucle.
- `continue` : passe à l'itération suivante.
- `pass` : instruction vide, utile comme bloc à définir plus tard.

7 Chaînes de caractères

- Déclaration : `'...'`, `"..."` ou `""" ... """` pour les chaînes multilignes.
- Immuables : une chaîne ne peut pas être modifiée après création.
- Exemple :

```
s = "bio"
# s[0] = "B"  --> Erreur : les strings sont immuables
```

8 Fonctions et Méthodes

Les fonctions et méthodes intégrées de Python permettent de manipuler efficacement les données, en particulier les chaînes de caractères, qui sont très utilisées en bioinformatique pour représenter des séquences biologiques.

8.1 Fonctions intégrées courantes

- `len(arg)` : retourne le nombre d'éléments de `arg`. Dans le cas d'une chaîne de caractères, elle retourne le nombre de caractères.
- `print(args[, sep=sepstr][, end=endstr])` : affiche les arguments passés en paramètre. Le paramètre `sep` définit le séparateur entre les éléments, et `end` définit la chaîne ajoutée à la fin de l'affichage.
- `input(string)` : affiche un message et permet à l'utilisateur de saisir une chaîne de caractères au clavier.

8.2 Méthodes courantes sur les chaînes de caractères

Les chaînes de caractères disposent de nombreuses méthodes permettant d'effectuer des opérations de recherche, de comptage et de transformation.

8.2.1 `count`

```
string1.count(string2[, start[, end]])
```

Cette méthode retourne le nombre d'occurrences de `string2` dans `string1`.

- `start` (optionnel) : position à partir de laquelle commence le comptage.
- `end` (optionnel) : position avant laquelle le comptage s'arrête.

8.2.2 `find`

```
string1.find(string2[, start[, end]])
```

Retourne la position de la première occurrence de `string2` dans `string1`. Si la chaîne recherchée n'est pas trouvée, la méthode retourne `-1`.

8.2.3 `startswith`

```
string1.startswith(string2[, start[, end]])
```

Retourne `True` si `string1` commence par `string2`, sinon `False`. Les paramètres `start` et `end` permettent de limiter la zone de comparaison.

8.2.4 `strip`, `lstrip` et `rstrip`

```
string1.strip([string2])
```

Supprime les caractères de `string2` au début et à la fin de `string1`. Si `string2` n'est pas spécifiée, les espaces sont supprimés par défaut.

- `lstrip` : supprime les caractères en début de chaîne.
- `rstrip` : supprime les caractères en fin de chaîne.

8.3 Autres méthodes utiles

— `str.find(x)` : retourne la position de `x` dans la chaîne (`-1` si absent).
— `str.count(x)` : retourne le nombre d'occurrences de `x`.
— `str.upper()` : convertit la chaîne en majuscules.
— `str.lower()` : convertit la chaîne en minuscules.
— `str.replace(a, b)` : remplace toutes les occurrences de `a` par `b`.

9 Structures de données

Les structures de données permettent de regrouper et d'organiser plusieurs valeurs au sein d'une même variable. En bioinformatique, elles sont très utilisées pour stocker des séquences biologiques, des résultats d'analyses ou des annotations.

9.1 Listes

Une **liste** est une collection ordonnée et modifiable d'éléments.

```
sequence = ["A", "T", "G", "C"]
```

Caractéristiques :
— Les éléments sont accessibles par leur indice.
— Les listes peuvent être modifiées après leur création.
— Elles peuvent contenir des éléments de types différents.

Exemple

```
sequence.append("A")
print(sequence)
```

9.2 Tuples

Un **tuple** est une collection ordonnée mais immuable.

```
position = (120, 450)
```

Un tuple est utilisé lorsque les données ne doivent pas être modifiées, par exemple pour représenter des coordonnées ou des paramètres fixes.

9.3 Dictionnaires

Un **dictionnaire** permet d'associer une clé à une valeur.

```
code_genetique = {"ATG": "Méthionine", "TAA": "Stop"}
```

Avantages :
— Accès rapide aux données par clé.
— Structure adaptée aux annotations biologiques.

9.4 Ensembles

Un **ensemble** est une collection non ordonnée d'éléments uniques.

```
nucleotides = {"A", "T", "G", "C"}
```

Les ensembles sont utiles pour supprimer les doublons et effectuer des comparaisons entre groupes de données.

10 Modules et Fichiers en Python

Pour écrire des programmes clairs, réutilisables et faciles à maintenir, Python permet de diviser le code en **modules** et de travailler avec des **fichiers**. Ces notions sont fondamentales en bioinformatique, où les projets peuvent rapidement devenir complexes.

10.1 Modules en Python

Un **module** est un fichier Python contenant des fonctions, des classes et des variables, pouvant être réutilisés dans d'autres programmes. Tout fichier avec l'extension `.py` peut être considéré comme un module.

10.1.1 Importation d'un module

Il existe plusieurs manières d'importer un module.

Importation complète

```
import math
print(math.sqrt(16))
```

Importation d'un élément spécifique

```
from math import sqrt
print(sqrt(16))
```

Importation avec alias

```
import numpy as np
```

L'utilisation d'un alias permet de simplifier l'écriture du code.

10.1.2 Modules standards

Python fournit une bibliothèque standard riche, incluant des modules couramment utilisés :

- `math` : fonctions mathématiques
- `sys` : interaction avec le système
- `os` : gestion des fichiers et répertoires
- `random` : génération de nombres aléatoires

10.1.3 Modules externes

Des modules spécialisés peuvent être installés via `pip` :

```
pip install biopython
```

Ces bibliothèques sont très utilisées en bioinformatique.

10.2 Création d'un module

Un module personnalisé peut être créé en écrivant des fonctions dans un fichier `.py`.

```
# fichier outils_bio.py
def longueur_seq(seq):
    return len(seq)
```

Puis importé dans un autre script :

```
import outils_bio
print(outils_bio.longueur_seq("ATGC"))
```

—

10.3 Fichiers en Python

La manipulation des fichiers est essentielle pour lire et écrire des données biologiques (fichiers FASTA, résultats d'analyses, journaux, etc.).

10.3.1 Ouverture et fermeture d'un fichier

```
f = open("sequence.txt", "r")
contenu = f.read()
f.close()
```

Modes courants :

- `r` : lecture
- `w` : écriture (écrase le contenu)
- `a` : ajout

10.3.2 Utilisation du mot-clé `with`

L'utilisation de `with` est recommandée car elle assure la fermeture automatique du fichier.

```
with open("sequence.txt", "r") as f:
    contenu = f.read()
```

10.3.3 Lecture ligne par ligne

```
with open("sequence.txt", "r") as f:
    for ligne in f:
        print(ligne.strip())
```

10.3.4 Écriture dans un fichier

```
with open("resultats.txt", "w") as f:
    f.write("Analyse terminée\n")
```

10.4 Bonnes pratiques

— Séparer le code en modules pour améliorer la lisibilité.

— Utiliser des noms explicites pour les fichiers et fonctions.

— Toujours fermer les fichiers ou utiliser `with`.

— Documenter les modules à l'aide de docstrings.

11 Documentation en Python

La documentation est essentielle pour comprendre comment utiliser une fonction ou une bibliothèque sans avoir à mémoriser tous les détails.

11.1 La fonction `help()`

La fonction `help()` permet d'afficher la documentation d'un objet.

```
help(len)
```

Elle indique :

— le rôle de la fonction,

— ses paramètres,

— sa valeur de retour.

```
print(str.upper.__doc__)
```

11.2 Docstrings

Les **docstrings** sont utilisées pour documenter le code écrit par le programmeur.

```
def longueur_sequence(seq):
    """Retourne la longueur d'une séquence ADN."""
    return len(seq)
```

—

12 Exécution d'un script Python

Un script Python est un fichier texte contenant des instructions Python, avec l'extension `.py`.

12.1 Exécution depuis le terminal

Pour exécuter un script :

```
python script.py
```

ou :

```
python3 script.py
```

12.2 Mode interactif

Python peut être utilisé en mode interactif pour tester rapidement des instructions.

```
python
>>> print("Hello Bioinformatics")
```

Ce mode est idéal pour l'apprentissage et les tests rapides.

12.3 Scripts exécutables sous Linux

Sous Linux, un script peut être rendu exécutable :

```
#!/usr/bin/env python3
```

Puis :

```
chmod +x script.py
./script.py
```

12.4 Environnements de développement

Les étudiants peuvent également utiliser des environnements de développement tels que :

- `VS Code`
- `PyCharm`
- `Jupyter Notebook`

Ces outils facilitent l'écriture, l'exécution et le débogage des scripts Python.